SENAI

Curso Superior de Análise e Desenvolvimento de Sistemas

Especificação dos Requisitos do Software e Análise do Projeto TrainUp

Versão 2.2

Equipe:

Ana Carolina Gonçalves

Bárbara Yasmin Pimenta dos Santos Tobias

Luana Gabrielle Ferreira Guedes Paes

Pedro Augusto Lombardi da Costa

Ruan Monteiro Brito

Prof.º Wesley Fioreze

Taubaté, 2026

Histórico das Revisões

Data	Versão	Descrição	Autor
03/08/2026	1.0	Abertura e triagem de demandas: criação do repositório com README e estrutura inicial	Pedro
10/08/2026	1.1	Escolha e delimitação da demanda: documento inicial com evidência da SAGA ativa, pesquisa e escopo do MVP	Ana; Bárbara; Luana; Pedro; Ruan
17/08/2026	1.2	Levantamento de requisitos: lista priorizada de RF, RNF e RN e matriz de integração das unidades curriculares	Bárbara; Ana
24/08/2026	1.3	Especificação textual dos fluxos dos requisitos funcionais e cenários de verificação dos requisitos não funcionais	Luana; Ruan
31/08/2026	1.4	Modelagem e arquitetura: diagramas de casos de uso, classes, DER e sequência, dicionário de dados e protótipo das telas	Ruan; Pedro; Luana; Ana
14/09/2026	1.5	Revisão e entrega da Sprint 1: consolidação final da ERS	Bárbara
19/10/2026	2.0	Atualização do documento conforme entregáveis da Sprint 2	Ana; Bárbara; Luana; Pedro; Ruan
24/10/2026	2.1	Revisão da Sprint 2: requisitos alinhados à implementação, tecnologias e evidências de testes nos apêndices	Ana; Bárbara; Luana; Pedro; Ruan
25/10/2026	2.2	Conferência final da Sprint 2: região da instância EC2, versões das ferramentas, referências às evidências (docs/api, backlog e changelog) e ajustes de formatação	Pedro

Sumário

1 Introdução **4**
1.1 Objetivos deste documento 4
1.2 Escopo do produto 4
1.2.1 Nome do produto e de seus componentes principais 5
1.2.2 Descrição do produto 5
1.2.3 Missão do produto 6
1.2.4 Tabela de viabilidade 6
1.2.5 Justificativa 7
1.3 Definições e siglas 7
1.4 Técnica(s) utilizada(s) para levantamento de requisitos 8
2 Requisitos Específicos **9**
2.1 Requisitos Funcionais (RF) 9
2.2 Requisitos Não Funcionais (RNF) 27
2.3 Regras de Negócio (RN) 32
2.4 Rastreabilidade: requisito, implementação e teste 33
3 Cronograma do Projeto **36**
3.1 Planejamento de Sprints 36
3.2 Controle de entregáveis — Sprint 1 (encerrada) 36
3.3 Controle de entregáveis — Sprint 2 (prazo final: 26/10/2026) 37
4 Protótipo do produto **38**
5 Diagramas UML **42**
5.1 Diagrama de Casos de Uso 42
5.2 Diagrama de Classes 44
5.3 Diagrama Entidade-Relacionamento (DER) 45
5.4 Diagramas de Sequência 46
5.4.1 Diagrama de Sequência — Executar sessão de treino [RF10 a RF12] 46
5.4.2 Diagrama de Sequência — Realizar check-in na catraca virtual [RF22] 47
6 Decisões de Projeto e Mudanças de Escopo **49**
6.1 Mudanças de escopo (Sprint 1 → Sprint 2) 49
6.2 Decisões técnicas e de produto 50
6.3 Dificuldades e riscos tratados 51
6.4 Limitações conhecidas 51
7 Casos de Testes **53**
7.1 Casos de teste 53
7.2 Resultado da execução 55
7.3 Registro das evidências 56
8 Apêndices **57**
Apêndice A — Dicionário de dados 57
Apêndice B — Tecnologias utilizadas 60
Apêndice C — Evidências dos testes 60

1 Introdução

Este documento apresenta a Especificação de Requisitos de Software (ERS) do TrainUp, solução desenvolvida pela equipe para a demanda “Serviços para Personal Trainer”, pesquisada e selecionada como ativa na Plataforma SAGA SENAI de Inovação. O problema identificado é a dificuldade de personal trainers autônomos e pequenos estúdios em administrar alunos, treinos e evolução física de forma manual (planilhas, papel ou aplicativos de mensagens), o que gera retrabalho, perda de histórico, dificuldade para gerar relatórios de evolução e para organizar a agenda e o controle de acesso dos atendimentos.

O TrainUp propõe um MVP (Produto Mínimo Viável) que permite ao Personal Trainer cadastrar e vincular alunos, montar fichas de treino, acompanhar a evolução de carga a partir das séries que o próprio aluno executa, gerar relatórios automáticos (com download em PDF) e organizar a agenda de aulas, enquanto o Aluno executa o treino pelo celular, acompanha sua evolução e realiza o check-in da aula em uma catraca virtual que simula um processo de automação industrial.

1.1 Objetivos deste documento

Este documento visa formalizar e detalhar de forma técnica e inequívoca todos os requisitos funcionais, não funcionais e restrições de projeto do sistema TrainUp. O objetivo principal é estabelecer um contrato técnico entre a equipe de desenvolvimento e os stakeholders, garantindo que as necessidades de negócio sejam traduzidas fielmente em funcionalidades de software, minimizando ambiguidades e riscos durante o ciclo de vida de desenvolvimento (SDLC).

Público Alvo: Stakeholders do projeto, analistas de sistemas, desenvolvedores de software, equipe de QA (Quality Assurance) e gestores de projeto.

1.2 Escopo do produto

O projeto TrainUp consiste no desenvolvimento de uma solução tecnológica voltada para a gestão de atendimentos de personal trainers e o acompanhamento da evolução de desempenho de seus alunos. O sistema abrange desde a camada de persistência de dados (PostgreSQL) até interfaces web responsivas para personal e aluno, integrando conceitos de Ciência de Dados (cálculo de evolução de carga, recordes pessoais, alertas de estagnação e relatórios), Automação Industrial (catraca virtual controlada por máquina de estados, com entradas e saídas de CLP, simulador de sensores e registro de eventos estilo

SCADA, sempre identificada como simulação) e Computação em Nuvem (contêineres Docker, configuração por variáveis de ambiente e desenvolvimento em instância AWS EC2 com code-server).

1.2.1 Nome do produto e de seus componentes principais

- TrainUp: núcleo do sistema.
- Módulo de Usuários e Vínculo: cadastro, autenticação, recuperação de senha e vínculo aluno–personal por código de convite com aprovação.
- Módulo de Treinos: catálogo de exercícios (base e biblioteca própria), fichas de treino por dia da semana e execução guiada da sessão de treino pelo aluno.
- Módulo de Evolução e Relatórios (Ciência de Dados): evolução de carga, recordes, alertas de estagnação, relatório consolidado por período, observação do personal e exportação em PDF (analytics e reports).
- Módulo de Agenda: agendamento, consulta, cancelamento e conclusão de aulas, com detecção de conflito de horário.
- Módulo de Automação (Catraca Virtual): máquina de estados da catraca, check-in por agendamento, anti-fraude, bloqueio, simulador de sensores, supervisório e histórico de acessos.
- API REST (FastAPI): camada de regras de negócio, autenticação JWT e documentação Swagger/OpenAPI.
- Front-end (React + Vite + Tailwind CSS): interface responsiva (menu lateral no desktop e barra inferior no celular).

1.2.2 Descrição do produto

O TrainUp é um sistema web para personal trainers organizarem o trabalho com seus alunos. De forma simples, é como uma “ficha de treino digital” que registra cada série feita pelo aluno, mostra se a carga está subindo e substitui planilhas e anotações soltas, além de controlar a entrada do aluno na academia pela agenda.

De forma técnica, trata-se de uma aplicação cliente-servidor com front-end em React consumindo uma API REST em FastAPI (versionada em /api/v1), que aplica as regras de negócio, autentica por JWT (hash de senha bcrypt) e persiste os dados em PostgreSQL por meio do SQLAlchemy, com migrations Alembic. A lógica de domínio mais complexa (indicadores, relatórios, resumo de sessão e máquina de estados da catraca) fica na camada de serviços. Todo o conjunto (banco, API e front-end servido por nginx) sobe com

Docker Compose e expõe apenas o nginx (porta 80 por padrão; na instância EC2 de desenvolvimento, onde a porta 80 é ocupada pelo code-server, o mapeamento é 8081:80). O desenvolvimento é feito em uma instância AWS EC2 com code-server (VS Code no navegador), o que evidencia o uso de Computação em Nuvem; a aplicação não depende de hospedagem em nuvem para funcionar.

O benefício central é transformar registros manuais e dispersos de treino em indicadores confiáveis e relatórios automáticos, apoiando a tomada de decisão do personal trainer (por exemplo, alertas de alunos sem evolução de carga) e a motivação do aluno (recordes pessoais e comparação com a sessão anterior).

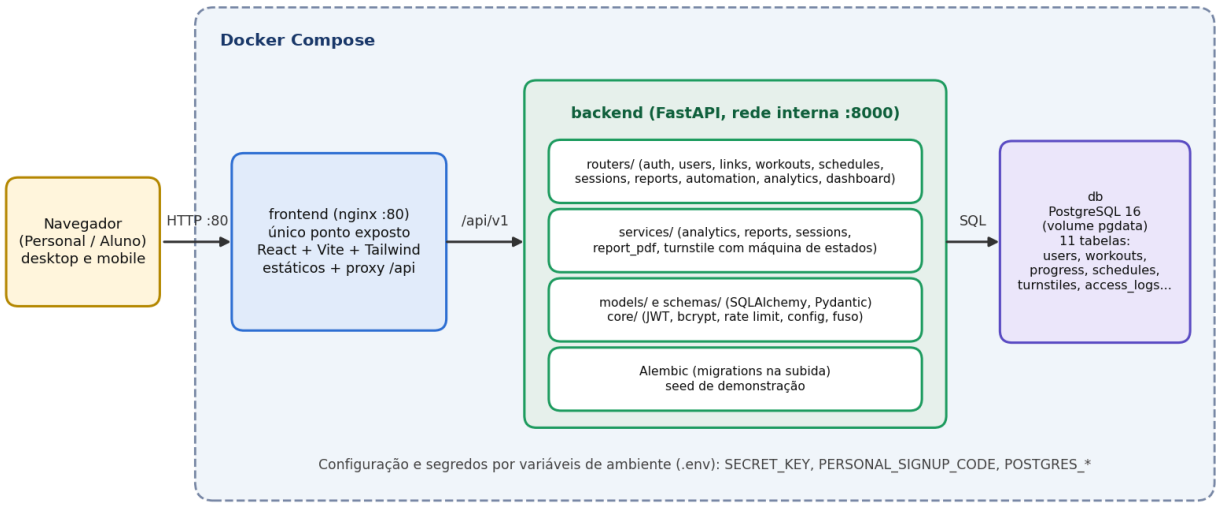


Figura 1 – Arquitetura do TrainUp (Docker Compose) – porta 80 (8081 na instância EC2).

1.2.3 Missão do produto

Prover uma solução digital eficiente, segura e responsiva para o acompanhamento de alunos de personal trainers, promovendo organização, rastreabilidade dos dados de treino e melhoria na comunicação entre profissional e aluno.

1.2.4 Tabela de viabilidade

Item	Descrição	Valor Unitário
Ambiente de desenvolvimento	Instância AWS EC2 (2 vCPU e 2 GB) na região N. Virginia (us-east-1) com code-server (VS Code) para o desenvolvimento, R$ 60,00 mensais	R$ 720,00
Armazenamento e backup	Volume EBS de 30 GB e cópia diária do banco PostgreSQL, R$ 15,00 mensais	R$ 180,00
Domínio	Registro anual .com.br	R$ 40,00

Item	Descrição	Valor Unitário
Desenvolvimento	Mão de obra estimada do MVP (back-end, front-end, testes e documentação)	R$ 2.000,00
Implantação	Configuração do ambiente (code-server e Docker Compose), migrations e seed, testes de aceitação e manual	R$ 600,00
Manutenção	Suporte, correções e atualizações, R$ 100,00 mensais a partir do segundo ano	R$ 1.200,00
Ferramentas	IDEs e plataformas	R$ 0,00

Valor total estimado do primeiro ano: R$ 3.540,00

Valor recorrente a partir do segundo ano: R$ 2.140,00 anuais

1.2.5 Justificativa

Os valores da tabela são estimativas e consideram custos operacionais mínimos e mão de obra técnica. O primeiro ano totaliza R$ 3.540,00, dos quais R$ 2.600,00 correspondem a custos únicos de desenvolvimento e implantação. A partir do segundo ano, o custo recorrente é de R$ 2.140,00 anuais, equivalentes a cerca de R$ 178,00 mensais. O produto possui potencial de escalabilidade, podendo ser comercializado como SaaS; os custos crescem de forma previsível com o armazenamento e com instâncias maiores, conforme o número de academias e alunos.

1.3 Definições e siglas

Número	Sigla	Definição
1	**ERS**	Especificação de Requisitos de Software.
2	**RF / RNF**	Requisito Funcional / Requisito Não Funcional.
3	**RN**	Regra de Negócio: normas e diretrizes que regem a lógica do domínio.
4	**CRUD**	Create, Read, Update, Delete (operações básicas de banco de dados).
5	**API**	Application Programming Interface.
6	**JWT**	JSON Web Token — padrão usado para autenticação e transporte de sessão do usuário.
7	**MVP**	Produto Mínimo Viável.
8	**CLP**	Controlador Lógico Programável: no TrainUp, a lógica de validação da catraca (`run_cycle`) simula o CLP.
9	**SCADA / IHM**	Sistema supervisório e interface homem-máquina: no TrainUp, o painel /automacao e a tabela `access_logs`.
10	**Catraca virtual**	Simulação de uma catraca de controle de acesso, com estado persistido no banco. Nunca é apresentada como equipamento real.
11	**FSM**	Máquina de estados finitos (TurnstileFSM), que valida as transições da catraca.
12	**Ficha de treino**	Treino de um aluno criado por um personal, composto por itens (exercício, séries, repetições e carga-alvo), opcionalmente associado a um dia da semana.

Número	Sigla	Definição
13	**Sessão de treino**	Execução de uma ficha pelo aluno; agrupa as séries registradas (tabela `progress`) entre o início e a finalização.
14	**Recorde / estagnação**	Recorde: carga máxima da sessão maior que todas as anteriores no exercício. Estagnação: exercício sem recorde há várias sessões (RN13).
15	**Código de convite**	Código de 8 caracteres de cada personal, usado pelo aluno para solicitar vínculo.
16	**Swagger**	Documentação interativa da API (OpenAPI) disponível em /api/docs.

1.4 Técnica(s) utilizada(s) para levantamento de requisitos

As seguintes técnicas foram utilizadas:

- Análise da demanda publicada na Plataforma SAGA SENAI de Inovação
- Entrevistas informais com personal trainers autônomos sobre a rotina de acompanhamento de alunos
- Observação de processos manuais (planilhas e aplicativos de mensagens)
- Análise de sistemas similares de gestão de estúdios/academias
- Revisão a cada Sprint: o feedback da demonstração e a execução real do sistema geraram os ajustes de requisitos registrados na seção 6 (por exemplo, a troca da evolução corporal manual pela evolução calculada das séries executadas).

Os registros encontram-se documentados no Documento Inicial da Demanda (Entregável 1 da Sprint 1), incluindo a evidência de captura de tela da demanda ativa na Plataforma SAGA SENAI.

2 Requisitos Específicos

2.1 Requisitos Funcionais (RF)

A lista abaixo apresenta a priorização dos 28 requisitos funcionais do TrainUp, revisados na Sprint 2 para refletir o sistema implementado (na Sprint 1 eram 16; as diferenças estão na seção 6.1). Todos os requisitos listados fazem parte do MVP entregue. Em seguida, cada requisito é detalhado com a estrutura de fluxos: identificador, descrição, ator, prioridade, gatilho, pré-condições, fluxo principal numerado, subfluxos/fluxos alternativos e dependências. Os caminhos de API citados partem de /api/v1.

ID	Nome	Prioridade	No MVP?
RF01	Registrar-se (Personal ou Aluno)	Alta	Sim
RF02	Autenticar-se (Login)	Alta	Sim
RF03	Recuperar senha	Média	Sim
RF04	Solicitar vínculo com Personal (código de convite)	Alta	Sim
RF05	Aprovar ou recusar solicitação de vínculo	Alta	Sim
RF06	Cadastrar Aluno (pelo Personal)	Alta	Sim
RF07	Consultar, editar e remover Alunos	Média	Sim
RF08	Gerenciar exercícios (catálogo base e biblioteca própria)	Alta	Sim
RF09	Gerenciar fichas de treino	Alta	Sim
RF10	Consultar fichas de treino (ficha do dia)	Alta	Sim
RF11	Executar sessão de treino e registrar séries	Crítica (núcleo)	Sim
RF12	Finalizar sessão e obter resumo (volume, comparação e recordes)	Crítica (núcleo)	Sim
RF13	Consultar sessões, resumos e evolução de carga	Crítica (núcleo)	Sim
RF14	Consultar alertas de estagnação	Alta	Sim
RF15	Consultar dicas de nutrição e recuperação	Baixa	Sim
RF16	Consultar relatório de evolução do aluno	Crítica (núcleo)	Sim
RF17	Registrar observação do Personal no relatório	Média	Sim
RF18	Baixar relatório em PDF	Alta	Sim
RF19	Agendar aula	Alta	Sim
RF20	Consultar agenda e cancelar aula	Alta	Sim
RF21	Concluir aula (Personal)	Média	Sim
RF22	Realizar check-in na catraca	Crítica (automação)	Sim
RF23	Confirmar passagem na catraca	Alta	Sim
RF24	Consultar estado da catraca e histórico de acessos	Alta	Sim
RF25	Simular cenários na catraca	Média	Sim
RF26	Rearmar catraca	Média	Sim
RF27	Consultar painel de visão geral do Personal	Média	Sim
RF28	Encerrar sessão (logout)	Alta	Sim

[RF01] — Registrar-se (Personal ou Aluno)

- **Descrição:** O sistema deve permitir que um visitante crie sua conta como Aluno ou como Personal Trainer. O cadastro de Personal exige um código de cadastro definido no ambiente (`PERSONAL_SIGNUP_CODE`); o Aluno pode informar o código de convite de um Personal para já solicitar o vínculo (RF04).
- **Ator principal:** Visitante (não autenticado).
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Usuário acessa a tela de login e escolhe "Cadastre-se" (ou abre o link /login?convite=<código> enviado pelo Personal, que já preenche o código de convite).
- **Pré-condições:** Nenhuma. Para Personal: possuir o código de cadastro.
- **Fluxo Principal:**
    1. O ator escolhe o perfil (Aluno ou Personal) e acessa o formulário de cadastro.
    2. O sistema apresenta os campos: nome, e-mail, senha e, conforme o perfil, objetivo e código de convite (Aluno, opcionais) ou código de cadastro (Personal).
    3. O ator preenche os campos e confirma (`POST /auth/register`).
    4. O sistema valida nome (mínimo 2 caracteres), e-mail e a política de senha (RN10) e, para Personal, o código de cadastro (RN09).
    5. O sistema verifica a unicidade do e-mail (RN10).
    6. O sistema cria o usuário com a senha armazenada em hash bcrypt; para Personal, gera um código de convite de 8 caracteres (RN01).
    7. Se o Aluno informou um código de convite válido, o sistema cria uma solicitação de vínculo com status PENDENTE (RF04).
    8. O sistema retorna 201 Created; o front-end autentica o ator (RF02) e o conduz ao seu painel.
- **Subfluxos / Fluxos Alternativos:**

    A1 — E-mail já cadastrado: erro 400 "E-mail já cadastrado"; nenhum usuário é criado.

    A2 — Senha fora da política (menos de 8 caracteres, sem letra ou sem número): erro 422 com a mensagem correspondente.

    A3 — Cadastro de Personal desabilitado (código de cadastro não configurado): erro 403.

    A4 — Código de cadastro de Personal incorreto: erro 403 "Código de cadastro de personal inválido".

    A5 — Código de convite inexistente (Aluno): erro 404 "Código de convite inválido"; a conta não é criada.
- **Dependências:** Nenhuma.

[RF02] — Autenticar-se (Login)

- **Descrição:** O sistema deve autenticar Personal ou Aluno por e-mail e senha, retornando um token JWT de sessão.
- **Ator principal:** Personal Trainer ou Aluno.

- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Usuário acessa a tela de login e envia as credenciais.
- **Pré-condições:** Possuir conta cadastrada (RF01 ou RF06).
- **Fluxo Principal:**
    1. O ator informa e-mail e senha.
    2. O sistema verifica o limite de tentativas falhas por IP + e-mail (RN08).
    3. O sistema localiza o usuário pelo e-mail e confere a senha com bcrypt (`POST /auth/login`).
    4. O sistema gera um token JWT assinado (HS256) com o identificador do usuário (`sub`), o perfil (`role`), a versão do token (tv, usada para revogação no logout e na redefinição de senha) e a expiração (480 minutos por padrão).
    5. O sistema retorna o token e os dados do usuário (incluindo o código de convite, se Personal, ou o nome do Personal vinculado, se Aluno).
    6. O front-end guarda o token e redireciona o ator para o painel do seu perfil (/personal ou /aluno).
- **Subfluxos / Fluxos Alternativos:**

    A1 — Credenciais inválidas: erro 401 “E-mail ou senha incorretos”, sem indicar qual dos dois falhou (segurança); a falha é contabilizada no limite de tentativas.

    A2 — Limite excedido: erro 429 após 5 falhas em 5 minutos para o mesmo IP + e-mail ou 20 falhas para o mesmo e-mail.

    A3 — Token ausente, inválido ou expirado em qualquer rota protegida: erro 401; o front-end limpa a sessão e volta ao login.
- **Dependências:** RF01, RF06.

[RF03] — Recuperar senha

- **Descrição:** O sistema deve permitir que o usuário defina uma nova senha por meio de um token temporário. No MVP não há envio de e-mail: o token é registrado no log do backend.
- **Ator principal:** Visitante (Personal ou Aluno que esqueceu a senha).
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Usuário escolhe “Esqueci minha senha” na tela de login.
- **Pré-condições:** Possuir conta cadastrada.
- **Fluxo Principal:**
    1. O ator informa o e-mail (`POST /auth/forgot`).
    2. O sistema gera um token aleatório, guarda apenas o hash SHA-256 e a validade (30 minutos) e registra o token no log do backend.
    3. O sistema responde com mensagem genérica, sem revelar se o e-mail existe.
    4. O ator informa o token e a nova senha (`POST /auth/reset`).
    5. O sistema valida o token e a validade, aplica a política de senha (RN10), grava o novo hash, invalida o token e incrementa token_version, o que derruba as sessões anteriores do usuário.

6. O sistema informa o sucesso e o ator retorna ao login.

- **Subfluxos / Fluxos Alternativos:**

  A1 — Token inválido ou expirado: erro 400 “Token inválido ou expirado”.

  A2 — Nova senha fora da política: erro 422.
- **Dependências:** RF01. Limitação do MVP: sem envio de e-mail (seção 6.4).

[RF04] — Solicitar vínculo com Personal (código de convite)

- **Descrição:** O Aluno sem vínculo solicita ser aluno de um Personal informando o código de convite dele. O vínculo só passa a valer após a aprovação do Personal (RF05).
- **Ator principal:** Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Aluno acessa o painel “Início” e informa o código de convite (ou o informa no cadastro, RF01).
- **Pré-condições:** Aluno autenticado e sem Personal vinculado.
- **Fluxo Principal:**
  1. O Aluno informa o código de convite (8 caracteres) e confirma (`POST /links/request`).
  2. O sistema verifica que o Aluno ainda não possui vínculo ativo (RN01).
  3. O sistema localiza o Personal pelo código.
  4. O sistema verifica que não há outra solicitação PENDENTE do mesmo Aluno para o mesmo Personal.
  5. O sistema cria a solicitação com status PENDENTE e retorna 201 Created.
  6. O Aluno acompanha o andamento de suas solicitações (`GET /links/mine`); o Personal as vê em “pendentes” (RF05).
- **Subfluxos / Fluxos Alternativos:**

  A1 — Aluno já vinculado: erro 400 “Você já está vinculado a um personal”.

  A2 — Código inexistente: erro 404 “Código de convite inválido”.

  A3 — Solicitação pendente duplicada: erro 400.
- **Dependências:** RF01, RF02.

[RF05] — Aprovar ou recusar solicitação de vínculo

- **Descrição:** O Personal analisa as solicitações pendentes e aceita ou recusa cada uma. Ao aceitar, o Aluno passa a ser aluno ativo do Personal.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal vê o aviso de solicitações pendentes no painel ou na tela “Alunos”.
- **Pré-condições:** Existir solicitação PENDENTE endereçada ao Personal.
- **Fluxo Principal:**
  1. O Personal abre a lista de solicitações pendentes (`GET /links/pending`).
  2. O Personal escolhe uma solicitação e opta por aceitar ou recusar (`POST /links/{id}/respond`).

3. Se aceitar, o sistema grava o Personal como responsável do Aluno e marca a solicitação como ACEITO (RN01).
4. O sistema cancela automaticamente as demais solicitações pendentes do mesmo Aluno.
5. O sistema registra a data da resposta e atualiza a lista de alunos do Personal.

- **Subfluxos / Fluxos Alternativos:**
  A1 — Recusar: o status passa a RECUSADO e o vínculo não é criado.
  A2 — Solicitação já respondida: erro 409 “Solicitação já respondida”.
  A3 — Aluno já vinculado a outro Personal: erro 409.
  A4 — Solicitação de outro Personal: erro 404.
- **Dependências:** RF04.

[RF06] — Cadastrar Aluno (pelo Personal)

- **Descrição:** O Personal cadastra diretamente um novo Aluno, que já nasce vinculado a ele (fluxo alternativo ao autocadastro com código de convite).
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Alunos” → “Novo aluno”.
- **Pré-condições:** Personal autenticado (RF02).
- **Fluxo Principal:**
  1. O Personal solicita a inclusão de um novo aluno.
  2. O sistema apresenta o formulário (nome, e-mail, senha inicial e objetivo).
  3. O Personal preenche e confirma (`POST /users`).
  4. O sistema valida os dados, a política de senha e a unicidade do e-mail (RN10).
  5. O sistema cria o usuário com perfil Aluno, senha em hash e vínculo ativo com o Personal autor da ação (RN01).
  6. O sistema exibe o aluno na lista de alunos do Personal.
- **Subfluxos / Fluxos Alternativos:**
  A1 — E-mail já cadastrado: erro 400.
  A2 — Campos inválidos: erro 422 / validação de formulário impede o envio.
  A3 — Usuário autenticado é Aluno: erro 403 “Apenas Personal Trainers”.
- **Observação:** O aluno recebe a senha inicial por fora do sistema (não há envio de e-mail no MVP).
- **Dependências:** RF02.

[RF07] — Consultar, editar e remover Alunos

- **Descrição:** O Personal lista seus alunos, consulta a ficha de cada um e pode remover a conta do aluno e seus dados; pela API, o Personal também pode alterar nome, objetivo ou senha do aluno (a interface do MVP oferece apenas listar, consultar e remover). O Aluno consulta e edita apenas o próprio cadastro e, pela API, pode remover a própria conta.
- **Ator principal:** Personal Trainer autenticado (Aluno, apenas para o próprio cadastro).

- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal acessa “Alunos” e abre a ficha do aluno.
- **Pré-condições:** O aluno deve estar vinculado ao Personal autenticado (RN02).
- **Fluxo Principal:**
    1. O Personal acessa a lista de alunos (`GET /users`).
    2. O Personal abre um aluno (`GET /users/{id}`); o sistema valida o acesso (RN02).
    3. Para editar (somente via API), o ator altera nome, objetivo ou senha e confirma (`PUT /users/{id}`); a alteração de senha por esta rota não revoga os tokens já emitidos (seção 6.4).
    4. Para remover, o Personal confirma a remoção (`DELETE /users/{id}`).
    5. O sistema apaga o aluno e seus dados associados (fichas, séries, sessões, agenda, observações, solicitações e logs de acesso) e responde 204.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Aluno não vinculado ao Personal: erro 404 “Usuário não encontrado” (não revela a existência do aluno).

    A2 — Tentativa de remover conta de Personal: erro 400 “No MVP apenas contas de aluno podem ser removidas”.

    A3 — O próprio Aluno remove a conta: pela API, DELETE /users/{id} com o próprio id apaga os mesmos dados e retorna 204; a interface não oferece essa ação.
- **Dependências:** RF05 ou RF06.

[RF08] — Gerenciar exercícios (catálogo base e biblioteca própria)

- **Descrição:** O sistema oferece um catálogo base de exercícios (somente leitura) e permite que cada Personal crie, edite e exclua os exercícios da sua própria biblioteca (nome, grupo muscular, descrição e link de vídeo opcional).
- **Ator principal:** Personal Trainer autenticado (Aluno apenas consulta).
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Exercícios”.
- **Pré-condições:** Personal autenticado (RF02).
- **Fluxo Principal:**
    1. O ator acessa a lista de exercícios (`GET /exercises`); o sistema retorna o catálogo base e os exercícios do Personal responsável (RN11).
    2. Para criar, o Personal informa os dados e confirma (`POST /exercises`).
    3. O sistema valida que não existe exercício com o mesmo nome na biblioteca visível ao Personal (RN11) e grava com o Personal como dono.
    4. Para editar (`PUT /exercises/{id}`) ou excluir (`DELETE /exercises/{id}`), o sistema valida que o exercício pertence ao Personal.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Nome duplicado: erro 400 “Já existe um exercício com esse nome na sua biblioteca”.

A2 — Edição ou exclusão do catálogo base: erro 403 “O catálogo base é somente leitura”.

A3 — Exercício de outro Personal: erro 404.

A4 — Exercício em uso por fichas ou históricos: erro 409 “Exercício em uso por fichas ou históricos”.

A5 — Aluno tenta criar exercício: erro 403.

- **Dependências:** RF02.

[RF09] — Gerenciar fichas de treino

- **Descrição:** O Personal cria, edita e exclui fichas de treino para os seus alunos. A ficha tem nome, dia da semana opcional e uma lista de itens (exercício, séries, repetições e carga-alvo).
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Treinos” → “Nova ficha” (ou edita/exclui uma existente).
- **Pré-condições:** Aluno vinculado ao Personal (RN02) e ao menos um exercício disponível.
- **Fluxo Principal:**
  1. O Personal seleciona o aluno e informa o nome e, opcionalmente, o dia da semana da ficha.
  2. O sistema apresenta os exercícios disponíveis (catálogo base e biblioteca do Personal).
  3. O Personal adiciona um ou mais itens, definindo exercício, séries, repetições e carga-alvo.
  4. O Personal confirma (`POST /workouts`, ou `PUT /workouts/{id}` na edição).
  5. O sistema valida o vínculo do aluno (RN02) e que todos os exercícios pertencem ao catálogo base ou ao Personal (RN11).
  6. O sistema grava a ficha e seus itens e a exibe na lista.
  7. Para excluir, o Personal confirma (`DELETE /workouts/{id}`) e o sistema remove a ficha e seus itens.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Aluno não vinculado ao Personal: erro 404 “Aluno não encontrado”.

  A2 — Exercício fora da biblioteca visível: erro 404.

  A3 — Ficha de outro Personal: erro 404 “Treino não encontrado”.

  A4 — Usuário autenticado é Aluno: erro 403.

  A5 — Ficha fora dos limites: ficha sem itens, nome com menos de 2 caracteres, séries fora de 1–20, repetições fora de 1–100 ou carga-alvo negativa: erro 422. O aluno da ficha não pode ser trocado na edição (a interface bloqueia o campo e a API ignora aluno_id no PUT).
- **Dependências:** RF06 ou RF05; RF08.

[RF10] — Consultar fichas de treino (ficha do dia)

- **Descrição:** Personal e Aluno consultam as fichas de treino. O Aluno visualiza a ficha do dia na tela “Treinar”: fichas associadas ao dia da semana atual ou, se não houver, as fichas sem dia definido.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Acesso a “Treinos” (Personal) ou “Treinar” / “Início” (Aluno).
- **Pré-condições:** O Aluno só vê as próprias fichas; o Personal só vê as fichas que criou (RN02).
- **Fluxo Principal:**
    1. O ator acessa a listagem (`GET /workouts`, opcionalmente filtrada por aluno) ou o treino do dia (`GET /workouts/today?weekday=n`).
    2. O sistema aplica o filtro de acesso (RN02).
    3. O sistema retorna as fichas, cada uma com seus itens e exercícios.
    4. O ator pode abrir uma ficha específica (`GET /workouts/{id}`).
- **Subfluxos / Fluxos Alternativos:**

    A1 — Ficha de outro usuário: erro 404 “Treino não encontrado”.

    A2 — Nenhuma ficha cadastrada: o sistema exibe estado vazio.

    Observação: GET /workouts/today considera sempre o aluno autenticado (para o Personal retorna lista vazia). Sem o parâmetro weekday o servidor usa o dia da semana em UTC; por isso a interface sempre envia o dia local (America/Sao_Paulo).
- **Dependências:** RF09.

[RF11] — Executar sessão de treino e registrar séries (núcleo do MVP)

- **Descrição:** O Aluno inicia uma sessão a partir da ficha do dia (a API também aceita sessão sem ficha, o “treino livre”, mas a interface sempre parte de uma ficha) e registra, série a série, repetições e carga. A tela oferece um temporizador de descanso configurável entre as séries (30, 45, 60, 90 ou 120 s; padrão 60 s).
- **Ator principal:** Aluno autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Aluno acessa “Treinar” e escolhe “Iniciar este treino”.
- **Pré-condições:** Aluno autenticado; a ficha, quando informada, deve ser do próprio Aluno (RN02).
- **Fluxo Principal:**
    1. O Aluno abre “Treinar”; o sistema exibe as fichas do dia (RF10).
    2. O Aluno inicia o treino (`POST /sessions`); o sistema cria a sessão com a data/hora de início e `finished_at` nulo.
    3. Para cada série concluída, o Aluno informa repetições e carga (`POST /sessions/{id}/sets`).
    4. O sistema valida os limites das séries (RN12), confirma que a sessão é do Aluno e que ainda está em andamento.

5. O sistema grava cada série em `progress`, vinculada à sessão, ao exercício e à ficha, e responde 201 com a quantidade salva.
6. O front-end inicia o temporizador de descanso (opcional) e segue para a próxima série.

- **Subfluxos / Fluxos Alternativos:**

  A1 — Ficha de outro aluno ou inexistente: erro 404 “Treino não encontrado”.

  A2 — Sessão inexistente ou de outro aluno: erro 404 “Sessão não encontrada”.

  A3 — Sessão já finalizada: erro 409 “Sessão já finalizada” (RN12).

  A4 — Exercício inexistente: erro 404.

  A5 — Repetições, carga ou número da série fora dos limites: erro 422.

  A6 — Usuário autenticado é Personal: erro 403 “Apenas alunos”.
- **Observação:** O registro avulso de séries sem sessão (`POST /progress`) permanece disponível na API, mas o fluxo guiado do front-end usa as sessões.
- **Dependências:** RF10.

[RF12] — Finalizar sessão e obter resumo (núcleo do MVP)

- **Descrição:** Ao finalizar o treino, o sistema fecha a sessão e devolve o resumo: total de séries, volume (carga × repetições), variação em relação à sessão anterior da mesma ficha, duração e recordes pessoais.
- **Ator principal:** Aluno autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Aluno escolhe “Finalizar treino”.
- **Pré-condições:** Sessão iniciada pelo próprio Aluno (RF11).
- **Fluxo Principal:**
  1. O Aluno finaliza o treino (`POST /sessions/{id}/finish`).
  2. O sistema grava `finished_at` com a data/hora atual, se ainda estiver nula.
  3. O serviço `session_summary` soma séries e volume da sessão.
  4. Para cada exercício, o serviço compara a carga máxima da sessão com a maior carga anterior do aluno e marca o recorde (RN13).
  5. O serviço localiza a sessão finalizada anterior da mesma ficha (em treino livre, a sessão finalizada anterior do aluno) e calcula a variação percentual de volume (não calculada se o volume anterior for zero).
  6. O sistema devolve o resumo (total de séries, volume, variação, exercícios, recordes e duração em minutos) e o front-end o exibe.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Sessão inexistente ou de outro aluno: erro 404 “Sessão não encontrada”.

  A2 — Sessão já finalizada: a data de término não é alterada e o resumo é devolvido novamente.

  A3 — Primeira sessão da ficha: não há sessão anterior; a variação de volume não é exibida.

A4 — Primeiro registro do exercício: não há carga anterior; nenhum recorde é marcado.

- **Observação:** O resumo também pode ser consultado depois pelo aluno ou pelo personal vinculado (`GET /sessions/{id}/summary`).
- **Dependências:** RF11.

[RF13] — Consultar sessões, resumos e evolução de carga (núcleo do MVP)

- **Descrição:** O sistema calcula, a partir do histórico real de séries, a evolução de carga por exercício (carga média e máxima, volume e progressão percentual) e permite listar as sessões e seus resumos.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Aluno acessa "Evolução"; Personal abre a ficha do aluno.
- **Pré-condições:** RN02: o Aluno só vê os próprios dados; o Personal só vê alunos vinculados.
- **Fluxo Principal:**
  1. O ator acessa a tela de evolução.
  2. O sistema valida o acesso (RN02) e resolve o aluno consultado.
  3. O sistema agrupa as séries por exercício e por dia (fuso America/Sao_Paulo, RN15) e calcula carga média, carga máxima, volume e número de séries por dia (`GET /analytics/evolution`).
  4. O sistema calcula a progressão percentual entre a carga média do primeiro e a do último dia de cada exercício (cada dia com séries do exercício conta como uma "sessão" nas análises).
  5. O ator pode listar as sessões (`GET /sessions`) e abrir o resumo de uma delas (`GET /sessions/{id}/summary`).
  6. O front-end apresenta os cartões e o gráfico de evolução de carga por exercício.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Nenhuma série registrada: o sistema exibe estado vazio.

  A2 — Aluno de outro Personal ou de outro aluno: erro 403 (Aluno) ou 404 (Personal).

  A3 — Personal sem informar `aluno_id`: erro 400 "Informe aluno_id".
- **Dependências:** RF11, RF12.

[RF14] — Consultar alertas de estagnação

- **Descrição:** O sistema identifica exercícios em que a carga máxima do aluno não bate recorde há várias sessões e gera um alerta com mensagem pronta para o personal revisar a ficha.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa "Visão geral" ou "Alunos"; Aluno acessa a própria evolução.
- **Pré-condições:** RN02. O exercício precisa ter ao menos 3 sessões registradas (RN13).

- **Fluxo Principal:**
    1. O ator solicita os alertas (`GET /analytics/stagnation`); o Personal, sem informar aluno, recebe os de todos os seus alunos.
    2. Para cada exercício, o sistema obtém a carga máxima de cada sessão (dia com séries do exercício) e conta quantas sessões se passaram desde o último recorde (RN13).
    3. Se a contagem atinge o limite configurado (`STAGNATION_N`, padrão 3, ou seja, 2 sessões sem recorde), o sistema cria o alerta.
    4. O sistema ordena os alertas do mais longo para o mais curto e devolve aluno, exercício, sessões sem recorde, carga atual e mensagem.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Sem exercícios estagnados: lista vazia.

    A2 — Exercício com menos de 3 sessões: não gera alerta.
- **Dependências:** RF13.

[RF15] — Consultar dicas de nutrição e recuperação

- **Descrição:** O sistema exibe dicas educativas gerais de nutrição e recuperação conforme o objetivo do aluno (ganho de massa, emagrecimento, resistência ou saúde geral), sempre acompanhadas de aviso para procurar nutricionista ou médico (RN14).
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Baixa / Sim (MVP).
- **Gatilho:** Aluno acessa “Evolução”; as dicas aparecem abaixo do relatório.
- **Pré-condições:** RN02.
- **Fluxo Principal:**
    1. O ator solicita as dicas (`GET /analytics/tips`).
    2. O sistema resolve o aluno e o objetivo cadastrado; se o objetivo não estiver mapeado, usa “saúde geral”.
    3. O sistema devolve três dicas do objetivo e o aviso de conteúdo educativo.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Aluno sem objetivo definido: são exibidas as dicas de saúde geral.
- **Dependências:** RF01 ou RF06.

[RF16] — Consultar relatório de evolução do aluno (núcleo do MVP)

- **Descrição:** O sistema gera, sob demanda, o relatório consolidado do aluno em um período: frequência às aulas, sessões de treino, desempenho por exercício, recordes, exercícios estagnados, volume semanal e resumo textual produzido por regras.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Personal acessa “Relatórios”; Aluno acessa “Evolução”.
- **Pré-condições:** RN02: o Aluno só acessa o próprio relatório; o Personal só de alunos vinculados.

- **Fluxo Principal:**
    1. O ator escolhe o aluno (Personal) e o período (7 dias, 30 dias, 3 meses ou datas personalizadas).
    2. O sistema valida o acesso (RN02) e o período (data inicial não posterior à final e no máximo 366 dias).
    3. O sistema calcula a frequência: aulas agendadas, realizadas (check-in ou concluídas), canceladas, faltas e presença percentual (falta = aula AGENDADO cujo horário já terminou; presença = realizadas ÷ (realizadas + faltas); aulas futuras contam como pendentes).
    4. O sistema calcula, por exercício: sessões no período, primeira/última/máxima carga, progressão percentual, volume total, recorde e estagnação (RN13). No relatório, a progressão usa a carga máxima do primeiro e do último dia do período, e a estagnação usa critério fixo (ao menos 3 sessões e 2 ou mais sem recorde), independente de STAGNATION_N.
    5. O sistema calcula o volume semanal e monta o resumo textual por regras.
    6. O sistema anexa a observação do Personal, se existir (RF17), e devolve o relatório (`GET /reports/aluno/{id}`).
- **Subfluxos / Fluxos Alternativos:**

    A1 — Período inválido: erro 400 “A data inicial deve ser anterior à final” ou “O período máximo é de 1 ano”.

    A2 — Nenhum treino no período: o resumo informa que o aluno não registrou treinos.

    A3 — Acesso não autorizado: erro 403 (Aluno) ou 404 (Personal).
- **Observação:** O relatório não é gravado: é recalculado a cada consulta, o que evita dados desatualizados (seção 6.1).
- **Dependências:** RF11, RF13, RF20.

[RF17] — Registrar observação do Personal no relatório

- **Descrição:** O Personal escreve uma observação ou recomendação (até 2.000 caracteres) que passa a aparecer no relatório do aluno, na tela e no PDF.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal acessa o relatório do aluno e edita a observação.
- **Pré-condições:** Aluno vinculado ao Personal (RN02).
- **Fluxo Principal:**
    1. O Personal escreve a observação e salva (`PUT /reports/aluno/{id}/note`).
    2. O sistema valida o vínculo (RN02) e o tamanho do texto.
    3. O sistema cria ou atualiza a observação única do aluno, registrando o Personal e a data de atualização.
    4. O relatório (RF16) e o PDF (RF18) passam a exibir a observação.

- **Subfluxos / Fluxos Alternativos:**
  - **A1 — Aluno não vinculado:** erro 404.
  - **A2 — Texto acima de 2.000 caracteres:** erro 422.
  - **A3 — Texto vazio:** a observação deixa de ser exibida no relatório.
  - **A4 — Usuário autenticado é Aluno:** erro 403.
- **Dependências:** RF16.

[RF18] — Baixar relatório em PDF

- **Descrição:** O sistema exporta o relatório do aluno em PDF (extensão opcional do RF16), com identificação do aluno e do personal, período, indicadores, gráficos, resumo e observação.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Ator escolhe “Baixar PDF” na tela de relatório/evolução.
- **Pré-condições:** RN02 e relatório disponível para o período (RF16).
- **Fluxo Principal:**
  1. O ator aciona “Baixar PDF” (`GET /reports/aluno/{id}/pdf`).
  2. O sistema valida o acesso e o período, monta o relatório (RF16) e o renderiza em PDF (reportlab).
  3. O sistema devolve o arquivo (Content-Disposition `relatorio-<aluno>-<início>_<fim>.pdf`); a interface o salva como relatorio-trainup-<início>_<fim>.pdf.
  4. O front-end inicia o download.
- **Subfluxos / Fluxos Alternativos:**
  - **A1 — Acesso não autorizado ou período inválido:** mesmos erros do RF16.
- **Dependências:** RF16.

[RF19] — Agendar aula

- **Descrição:** O Aluno agenda uma aula para si com o seu Personal; o Personal agenda uma aula para um aluno vinculado. O sistema impede sobreposição de horários do mesmo Personal.
- **Ator principal:** Aluno ou Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Ator acessa “Agenda” e escolhe data, hora e duração.
- **Pré-condições:** Aluno com Personal vinculado (RN01) ou Personal com aluno vinculado (RN02).
- **Fluxo Principal:**
  1. O ator informa data/hora de início (fuso America/Sao_Paulo) e duração (15 a 240 minutos); o Personal também escolhe o aluno.
  2. O front-end converte a data local para UTC e envia (`POST /schedules`).
  3. O sistema identifica o aluno e o personal da aula.

4. O sistema valida que o horário não está no passado, com tolerância de 5 minutos (RN03).
5. O sistema verifica sobreposição com as aulas AGENDADO ou CHECKIN do Personal (RN03).
6. O sistema cria a aula com status AGENDADO e retorna 201 Created.

- **Subfluxos / Fluxos Alternativos:**

  A1 — Aluno sem Personal vinculado: erro 400 “Vincule-se a um personal antes de agendar”.

  A2 — Horário no passado: erro 400 “Escolha um horário futuro”.

  A3 — Conflito de horário: erro 409 “Horário indisponível para este personal”.

  A4 — Duração fora de 15–240 minutos: erro 422.

  A5 — Personal informa aluno não vinculado: erro 404.

  A6 — Personal sem informar o aluno ou Aluno informando outro aluno: erro 400 “Informe aluno_id” (Personal) ou 403 (Aluno). A interface oferece durações de 30, 45, 60 e 90 minutos e horários em passos de 15 minutos; a API aceita de 15 a 240.
- **Dependências:** RF05 ou RF06.

[RF20] — Consultar agenda e cancelar aula

- **Descrição:** Aluno e Personal consultam suas aulas (agenda semanal) e podem cancelar uma aula ainda AGENDADA.
- **Ator principal:** Aluno ou Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Ator acessa “Agenda” e aciona “Cancelar” em uma aula.
- **Pré-condições:** O ator deve ser o aluno ou o personal da aula (RN05).
- **Fluxo Principal:**
  1. O ator abre a agenda (`GET /schedules`); o sistema retorna as aulas do ator ordenadas por horário.
  2. O ator escolhe cancelar uma aula (`PATCH /schedules/{id}/status` com `CANCELADO`).
  3. O sistema valida o acesso (RN05) e a transição de estado (RN04).
  4. O sistema grava o status CANCELADO e atualiza a agenda.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Aula de outra pessoa: erro 404 “Agendamento não encontrado”.

  A2 — Transição inválida (aula já em CHECKIN, CONCLUIDO ou CANCELADO): erro 409 “Transição inválida”.

  A3 — Aluno tenta outro status que não seja cancelar: erro 403 “O aluno só pode cancelar a própria aula”.

  A4 — Status diferente de CANCELADO ou CONCLUIDO: (por exemplo CHECKIN) erro 422; o check-in só ocorre pela catraca (RN04).
- **Dependências:** RF19.

[RF21] — Concluir aula (Personal)

- **Descrição:** Depois do check-in, o Personal marca a aula como concluída.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal aciona “Concluir” em uma aula com check-in feito.
- **Pré-condições:** A aula deve estar em CHECKIN (RN04) e pertencer ao Personal.
- **Fluxo Principal:**
    1. O Personal escolhe concluir a aula (`PATCH /schedules/{id}/status` com `CONCLUIDO`).
    2. O sistema valida o acesso (RN05) e a transição CHECKIN → CONCLUIDO (RN04).
    3. O sistema grava o status CONCLUIDO; a aula passa a contar como realizada nos relatórios (RF16).
- **Subfluxos / Fluxos Alternativos:**

    A1 — Aula ainda AGENDADO, cancelada ou já concluída: erro 409 “Transição inválida”.

    A2 — Aluno tenta concluir: erro 403.
- **Dependências:** RF22.

[RF22] — Realizar check-in na catraca (núcleo da automação)

- **Descrição:** O Aluno faz o check-in da aula agendada na catraca virtual. O sistema simula o ciclo sensor → CLP → atuador → SCADA: valida o agendamento e a janela de acesso e, se permitido, libera a catraca.
- **Ator principal:** Aluno autenticado.
- **Prioridade / MVP:** Crítica / Sim (automação industrial simulada).
- **Gatilho:** Aluno aciona “Fazer check-in” em uma aula agendada (na tela Início ou nos detalhes da aula em Agenda).
- **Pré-condições:** Aula AGENDADO do próprio aluno (RF19).
- **Fluxo Principal:**
    1. O Aluno aciona o check-in da aula (`POST /automacao/checkin`).
    2. O sistema atualiza o estado da catraca pelas transições por tempo (timeout de passagem e rearme automático).
    3. O sistema localiza o agendamento; se não existir ou não for do aluno, o acesso será negado.
    4. Se a catraca estiver em BLOQUEIO, a leitura é ignorada e o acesso é negado (RN07).
    5. Caso contrário, a catraca passa de TRAVADA para VALIDANDO (LED amarelo) e o CLP valida: agendamento existente, status AGENDADO e hora atual dentro da janela de 15 minutos antes do início até o fim da aula (RN06).
    6. Se válido: LIBERADA (solenoide e LED verde) e, em seguida, AGUARDANDO_PASSAGEM por 10 segundos; as negativas seguidas são zeradas; a aula vai para CHECKIN com `checkin_at` (RN04).

7. O sistema grava o evento em `access_logs` com o tempo de ciclo e o detalhe linha a linha, atualiza a catraca e a aula e devolve `granted`, motivo, transições, log do ciclo e estado da catraca.
8. O front-end exibe a catraca liberada ou o acesso negado com o motivo.

- **Subfluxos / Fluxos Alternativos:**

  A1 — Sem agendamento / credencial não reconhecida: acesso negado; a catraca volta a TRAVADA e conta uma negativa.

  A2 — Cedo demais ou aula encerrada: acesso negado com o motivo "fora da janela".

  A3 — Check-in já realizado (anti-fraude): acesso negado "check-in já realizado para esta aula".

  A4 — Aula com status diferente de AGENDADO (cancelada, concluída): acesso negado.

  A5 — Três negativas seguidas: a catraca vai para BLOQUEIO com sirene por 60 segundos ou até rearme manual (RF26, RN07).

  A6 — Novo check-in durante a espera de passagem: a espera anterior é encerrada e um novo ciclo é iniciado.

  Observação: a interface do Aluno usa a primeira catraca cadastrada (Catraca 01); a API aceita turnstile_id.
- **Dependências:** RF19.

[RF23] — Confirmar passagem na catraca

- **Descrição:** Após a liberação, a passagem do aluno (sensor óptico I0.1) é confirmada e a trava é religada, e só o aluno liberado ou o personal responsável por ele pode confirmá-la (RN02). Sem a confirmação, a trava religa sozinha ao fim do tempo de passagem.
- **Ator principal:** Aluno liberado ou o seu Personal Trainer, autenticados (simulação do sensor).
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** O aluno liberado aciona "Já passei pela catraca" (Início/Agenda) ou o seu personal aciona "Registrar passagem (I0.1)" (tela Catraca).
- **Pré-condições:** Catraca em AGUARDANDO_PASSAGEM.
- **Fluxo Principal:**
  1. O ator registra a passagem (`POST /automacao/passagem`).
  2. O sistema atualiza o estado por tempo e verifica que há passagem pendente.
  3. O sistema confere que a liberação pertence ao usuário: o aluno liberado ou o personal responsável por ele (RN02).
  4. O sistema aplica a transição AGUARDANDO_PASSAGEM → TRAVADA e limpa o prazo de liberação.
  5. O sistema grava o evento PASSAGEM em `access_logs` e devolve o novo estado.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Sem passagem pendente: erro 409 "Sem passagem pendente".

A2 — Tempo de passagem esgotado (10 s): a catraca volta a TRAVADA automaticamente e o sistema registra o evento TIMEOUT (RN09).

A3 — Liberação de outro usuário: erro 403 “Esta liberação pertence a outro usuário”; a catraca continua aguardando a passagem do aluno liberado.

- **Dependências:** RF22.

[RF24] — Consultar estado da catraca e histórico de acessos

- **Descrição:** O sistema mostra o estado atual das catracas (semáforo, entradas e saídas do CLP, negativas seguidas e tempo restante) e o histórico de acessos com filtros. O Personal enxerga os acessos dos seus alunos; o Aluno, apenas os próprios.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Catraca” (rota /automacao); Aluno acompanha o estado e o resultado do check-in no cartão de check-in (Início e Agenda).
- **Pré-condições:** Usuário autenticado.
- **Fluxo Principal:**
  1. O ator consulta as catracas (`GET /automacao/turnstiles`) e o estado de uma delas (`GET /automacao/status`).
  2. O sistema atualiza o estado por tempo e devolve o estado, a cor do semáforo, o mapa de E/S (I0.0–I0.2 e Q0.0–Q0.4), as negativas seguidas e, para o Personal, as estatísticas do dia (permitidos, negados e ciclo médio).
  3. O ator consulta o histórico (`GET /automacao/logs`), filtrando por catraca, resultado (permitido/negado), evento e dia.
  4. O sistema restringe os registros ao Personal ou ao Aluno (RN02) e oculta o detalhe técnico do ciclo para o Aluno.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Catraca inexistente: erro 404 “Catraca não encontrada”.

  A2 — Sem registros: lista vazia.
- **Observação:** O painel atualiza o estado por consulta periódica (polling a cada 2 s).
- **Dependências:** RF22.

[RF25] — Simular cenários na catraca

- **Descrição:** O Personal dispara cenários de teste da catraca, sem depender de dados preparados: credencial válida, credencial inválida, aluno sem aula, fora da janela e tentativa dupla. Os eventos são registrados como simulados (RN16).
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal escolhe um cenário no “Simulador de sensores”.
- **Pré-condições:** Ao menos um aluno vinculado ao Personal.
- **Fluxo Principal:**
  1. O Personal escolhe o cenário e a catraca (`POST /automacao/simulate`).

2. O sistema escolhe o aluno (informado pela API ou, como faz a interface, o primeiro do Personal) e monta um agendamento fictício conforme o cenário, sem gravar aula.
3. O sistema executa o ciclo da catraca (RF22) uma vez ou, na tentativa dupla, duas vezes com o mesmo agendamento.
4. O sistema grava os eventos em `access_logs` com `simulated = true` e o nome do cenário (RN16).
5. O sistema devolve os ciclos executados e o painel anima as transições de estado.

- **Subfluxos / Fluxos Alternativos:**

  A1 — Personal sem alunos: erro 400 “Cadastre ao menos um aluno para simular”.

  A2 — Aluno tenta usar o simulador: erro 403.

  A3 — Cenário inexistente: erro 422.
- **Dependências:** RF22.

[RF26] — Rearmar catraca

- **Descrição:** O Personal supervisor rearma manualmente a catraca em BLOQUEIO (botão I0.2). O rearme também ocorre automaticamente após 60 segundos.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal aciona “Rearmar (I0.2)”.
- **Pré-condições:** Catraca em BLOQUEIO.
- **Fluxo Principal:**
  1. O Personal aciona o rearme (`POST /automacao/reset`).
  2. O sistema valida que a catraca está em BLOQUEIO.
  3. O sistema aplica BLOQUEIO → TRAVADA, zera as negativas seguidas e desliga a sirene.
  4. O sistema grava o evento RESET em `access_logs` e devolve o novo estado.
- **Subfluxos / Fluxos Alternativos:**

  A1 — Catraca fora de BLOQUEIO: erro 409 “a catraca não está em BLOQUEIO”.

  A2 — Rearme automático: após 60 s o estado volta a TRAVADA e o sistema registra RESET “rearme automático após bloqueio”.

  A3 — Aluno tenta rearmar: erro 403.
- **Dependências:** RF22.

[RF27] — Consultar painel de visão geral do Personal

- **Descrição:** Tela inicial do Personal que resume, em um só lugar, indicadores já calculados por outros requisitos: alunos ativos, aulas de hoje, alertas de estagnação, solicitações de vínculo pendentes e próximas aulas. O Aluno tem uma tela “Início” equivalente, com a ficha do dia, a próxima aula (com o cartão de check-in) e, enquanto não houver vínculo, o formulário para solicitar vínculo por código de convite (RF04).
- **Ator principal:** Personal Trainer autenticado (Aluno, na tela “Início”).
- **Prioridade / MVP:** Média / Sim (MVP).

- **Gatilho:** Acesso à tela inicial após o login.
- **Pré-condições:** Usuário autenticado.
- **Fluxo Principal:**
    1. O front-end solicita o painel (`GET /dashboard/personal`).
    2. O sistema conta os alunos vinculados, as aulas do dia (exceto canceladas), as solicitações pendentes e os alertas de estagnação (RF14).
    3. O sistema devolve também os cinco alertas mais longos e as próximas seis aulas (AGENDADO ou CHECKIN com início a partir de uma hora atrás).
    4. O front-end exibe os cartões, a lista de próximas aulas, os alertas e o aviso de solicitações pendentes.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Usuário autenticado é Aluno: o endpoint do painel do Personal retorna 403; a tela “Início” do Aluno usa as fichas (RF10) e a agenda (RF20).
- **Dependências:** RF05, RF14, RF19.

[RF28] — Encerrar sessão (logout)

- **Descrição:** O sistema deve permitir que Personal Trainer ou Aluno encerre a sessão autenticada, revogando os tokens JWT anteriormente emitidos para a conta.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** O usuário aciona “Sair” na interface.
- **Pré-condições:** Usuário autenticado com token válido.
- **Fluxo Principal:**
    1. O ator aciona o comando de logout (`POST /auth/logout`).
    2. O sistema incrementa `token_version` do usuário e persiste a alteração.
    3. O sistema responde 204 No Content; o front-end encerra a sessão local e retorna ao login.
- **Subfluxos / Fluxos Alternativos:**

    A1 — Token ausente, inválido ou já revogado: erro 401.
- **Dependências:** RF02.

2.2 Requisitos Não Funcionais (RNF)

Estipulam as qualidades e restrições técnicas do sistema. A tabela a seguir prioriza os 8 requisitos não funcionais do TrainUp; em seguida, cada um é detalhado como um cenário de verificação, com categoria, componente afetado, condições iniciais, procedimento de verificação, métrica, resultado esperado, evidência a coletar e a situação atual da evidência (seção 7).

ID	Categoria	Prioridade	Situação da evidência
RNF01	Segurança (autenticação/senha)	Alta	Verificado por testes (7.1)
RNF02	Confidencialidade / controle de acesso	Crítica	Verificado por testes (7.1)
RNF03	Tempo de resposta	Alta	Verificado (7.1 a 7.3)

ID	Categoria	Prioridade	Situação da evidência
RNF04	Usabilidade / Responsividade	Alta	Verificado (7.1 a 7.3)
RNF05	Computação em Nuvem (ambiente de desenvolvimento)	Média	Verificado (7.1 a 7.3)
RNF06	Confiabilidade dos dados (histórico apenas incremental)	Alta	Verificado por teste (7.1)
RNF07	Manutenibilidade (arquitetura em camadas e migrations)	Média	Verificado (7.1 a 7.3)
RNF08	Confiabilidade e desempenho da automação (catraca)	Alta	Verificado (7.1 a 7.3)

[RNF01] — Segurança de autenticação

- **Categoria:** Segurança.
- **Descrição:** Senhas nunca são armazenadas em texto puro; toda rota autenticada exige um JWT válido; falhas repetidas de login são limitadas por IP + e-mail e também por conta; logout e redefinição de senha por token (RF03) revogam os tokens JWT anteriores (a alteração por PUT /users/{id} não revoga, seção 6.4); a política de senha exige 8 caracteres com letras e números.
- **Componente / processo afetado:** `app/core/security.py`, `app/core/ratelimit.py`, `app/routers/auth.py` (back-end).
- **Condições iniciais:** Ambiente de demonstração com banco populado (`seed.py`).
- **Procedimento de verificação:** (1) Inspecionar a coluna `hashed_password` no banco e confirmar que não é texto plano; (2) chamar um endpoint protegido (ex.: `GET /users`) sem o cabeçalho Authorization e depois com um token inválido; (3) enviar senha errada ao login; (4) cadastrar usuário com senha fraca.
- **Métrica / parâmetro:** Hash bcrypt (prefixo $2b$); resposta 401 sem token válido; 422 para senha fora da política; 429 após 5 falhas em 5 minutos por IP + e-mail ou 20 falhas por e-mail, mesmo trocando o IP.
- **Resultado esperado:** Senha sempre em hash; requisição sem token válido retorna 401 Unauthorized.
- **Evidência a coletar:** Testes test_senhas_do_seed_estao_em_bcrypt, test_troca_de_senha_derruba_sessoes_antigas, test_rate_limit_do_login, test_rate_limit_por_email_ignora_ip_forjado, test_x_real_ip_ignorado_sem_proxy_confiavel, test_senha_fraca_e_rejeitada e test_logout_revoga_o_token.

[RNF02] — Confidencialidade / controle de acesso por perfil

- **Categoria:** Segurança / Confidencialidade.
- **Descrição:** Um Aluno nunca acessa dados de outro Aluno; um Personal nunca acessa alunos não vinculados a ele; ações de Personal são negadas ao Aluno (RN02).
- **Componente / processo afetado:** `resolve_aluno`, `require_personal` e `require_aluno` em `app/core/security.py`, usados por todos os routers.
- **Condições iniciais:** Dois alunos de demonstração distintos (Ana e Bruno), autenticados com tokens diferentes.

- **Procedimento de verificação:** Autenticar como a Aluna Ana e tentar consultar `GET /analytics/evolution?aluno_id=3` e `GET /reports/aluno/3` (dados do Bruno), e `POST /exercises`.
- **Métrica / parâmetro:** Resposta HTTP 403 Forbidden.
- **Resultado esperado:** Acesso negado, com mensagem explícita (“Acesso negado” / “Apenas Personal Trainers”).
- **Evidência a coletar:** Testes test_personal_nao_acessa_aluno_de_outro, test_aluno_so_ve_o_proprio_relatorio e test_aluno_nao_cria_exercicio_nem_edita_catalogo_base e test_passagem_so_pelo_aluno_liberado_ou_seu_personal.

[RNF03] — Tempo de resposta

- **Categoria:** Desempenho.
- **Descrição:** Em ambiente de demonstração, a consulta de evolução de carga (RF13) deve responder em até 3 segundos.
- **Componente / processo afetado:** `GET /api/v1/analytics/evolution`.
- **Condições iniciais:** Banco populado pelo seed (aproximadamente 6 meses de histórico por aluno).
- **Procedimento de verificação:** Medir o tempo de resposta da rota em 5 chamadas consecutivas com os dados do seed, usando docs/evidencias/coletar_evidencias_api.py; o teste test_rnf03_evolucao_responde_em_ate_3_segundos repete a medição na suíte automatizada.
- **Métrica / parâmetro:** Tempo de resposta ≤ 3 segundos.
- **Resultado esperado:** Todas as 5 chamadas dentro do limite.
- **Evidência a coletar:** Saída em docs/evidencias/evidencias-api.txt (execução de 24/10/2026: média de 18,5 ms e máximo de 51,5 ms em 5 chamadas, contra o limite de 3 s) e teste test_rnf03_evolucao_responde_em_ate_3_segundos.

[RNF04] — Usabilidade e responsividade

- **Categoria:** Usabilidade.
- **Descrição:** A interface deve ser utilizável em telas de desktop e de celular, com mensagens claras de erro e sucesso, contraste adequado (AA) e navegação adaptada (menu lateral no desktop e barra inferior no celular).
- **Componente / processo afetado:** Front-end React + Tailwind (todas as páginas).
- **Condições iniciais:** Aplicação em execução com o banco de demonstração.
- **Procedimento de verificação:** Abrir as telas do Personal e do Aluno em 1440 px (desktop) e 375 px (celular), verificar a rolagem horizontal e provocar um erro de validação (login com senha errada); o script docs/evidencias/capturar_telas.py automatiza as capturas e a verificação.
- **Métrica / parâmetro:** Nenhum elemento cortado ou sobreposto nas duas resoluções; mensagem de erro visível e compreensível.

- **Resultado esperado:** Layout adaptado e mensagem de erro clara (“E-mail ou senha incorretos”).
- **Evidência a coletar:** 26 capturas de tela (13 telas × 2 resoluções) em docs/evidencias/prints e o relatório rnf04-overflow.txt (24/10/2026), que não encontrou nenhuma tela com rolagem horizontal; teste automatizado Login.test.jsx para a mensagem de erro.

[RNF05] — Computação em Nuvem (ambiente de desenvolvimento)

- **Categoria:** Computação em Nuvem.
- **Descrição:** O projeto deve ser desenvolvido em ambiente de nuvem (instância AWS EC2 com code-server) e estar preparado para execução com Docker Compose, configuração por variáveis de ambiente e nenhum segredo no código-fonte; em produção, a API não inicia sem `SECRET_KEY` de pelo menos 32 caracteres e somente a porta do nginx fica exposta (80 por padrão; 8081 na instância EC2, onde a 80 é usada pelo code-server).
- **Componente / processo afetado:** `docker-compose.yml`, `.env.example`, `app/core/config.py`, Dockerfiles e nginx.
- **Condições iniciais:** Repositório do projeto publicado no GitHub; instância AWS EC2 (Amazon Linux 2023) com Docker e code-server; porta 80 ocupada pelo code-server e porta 8081 liberada no Security Group para o front-end.
- **Procedimento de verificação:** Inspecionar o código e confirmar a ausência de credenciais fixas; subir com `docker compose up -d --build`; acessar o VS Code (code-server) da instância, o front-end pela porta do nginx (80 por padrão; 8081 na EC2, em http://<IP-da-instância>:8081) e o Swagger em /api/docs.
- **Métrica / parâmetro:** Nenhuma credencial hardcoded; aplicação acessível pela porta do nginx (80 por padrão; 8081 na EC2) com o back-end na rede interna.
- **Resultado esperado:** Projeto desenvolvido e executado no ambiente de nuvem, com front-end e back-end comunicando-se corretamente.
- **Evidência a coletar:** Arquivo `.env.example`, `docker-compose.yml` e captura do code-server na instância EC2 com a aplicação em execução.
- **Situação:** Verificado. A configuração por variáveis de ambiente e o Docker Compose estão prontos, o passo a passo de execução está no README e a captura do code-server na instância EC2 com a aplicação em execução está em docs/evidencias/prints/14-ec2-code-server.png. Nesse ambiente a porta 80 é ocupada pelo code-server; por isso o front-end é publicado em 8081 (mapeamento 8081:80 no docker-compose.yml, apenas na instância; o repositório mantém 80:80) e a captura exibe o acesso por http://<IP>:8081. O princípio de exposição única do nginx é mantido. Os testes test_producao_exige_secret_key, test_producao_recusa_secret_key_curta e test_producao_aceita_secret_key_forte comprovam que a API recusa iniciar em produção sem SECRET_KEY (ou com chave curta). Não há segredo de produção fixo no código; as exceções são a senha das contas de demonstração do seed (SEED_DEMO) e os valores padrão de desenvolvimento do PostgreSQL no docker-compose.yml, que devem ser trocados fora da demonstração. O Swagger (/api/docs) só fica disponível com ENABLE_DOCS=true (padrão).

[RNF06] — Confiabilidade dos dados de treino (histórico apenas incremental)

- **Categoria:** Confiabilidade.
- **Descrição:** O histórico de séries e de acessos é apenas incremental: não há rota de alteração ou exclusão de séries (`progress`) nem de eventos (`access_logs`), e uma sessão finalizada não aceita novas séries. A evolução e os relatórios são recalculados sempre a partir desse histórico.
- **Componente / processo afetado:** `app/routers/progress.py`, `app/routers/sessions.py`, tabelas `progress` e `access_logs`.
- **Condições iniciais:** Aluno com ao menos uma sessão finalizada.
- **Procedimento de verificação:** Inspecionar as rotas expostas no Swagger (somente POST e GET para séries e logs) e tentar `POST /sessions/{id}/sets` em sessão finalizada.
- **Métrica / parâmetro:** Zero endpoints de UPDATE/DELETE para `progress` e `access_logs`; resposta 409 para série em sessão finalizada.
- **Resultado esperado:** Histórico preservado; nova série em sessão finalizada é rejeitada.
- **Evidência a coletar:** Teste test_serie_em_sessao_finalizada_e_rejeitada, e inspeção das rotas no Swagger (progress expõe apenas POST e GET; access_logs, apenas leitura). A exclusão de um aluno remove os seus dados em cascata (RF07), única exceção prevista.

[RNF07] — Manutenibilidade (arquitetura em camadas e migrations)

- **Categoria:** Manutenibilidade.
- **Descrição:** O back-end separa rotas (`routers/`), regras de domínio complexas (`services/`), modelos (`models/`) e contratos de entrada/saída (`schemas/`); o esquema do banco é versionado com Alembic e há testes automatizados (pytest e vitest) executados a cada push por integração contínua (GitHub Actions: ruff, pytest, vitest, build e docker compose).
- **Componente / processo afetado:** Pasta `backend/app` (routers, services, models, schemas, core) e `backend/alembic`.
- **Condições iniciais:** Código-fonte disponível no repositório.
- **Procedimento de verificação:** Inspecionar um fluxo completo (ex.: RF12) e confirmar que a rota (routers/sessions.py) apenas valida o acesso e delega o cálculo ao serviço session_summary; executar pytest e verificar as migrations 0001_baseline e 0002_token_version.
- **Métrica / parâmetro:** Cálculos de indicadores, relatórios, resumo de sessão e máquina de estados fora dos routers; 61 testes de back-end (95% de cobertura) e 64 de front-end aprovados.
- **Resultado esperado:** Separação de responsabilidades confirmada por leitura de código e testes passando.
- **Evidência a coletar:** Saída do pytest (seção 7.2), migrations 0001_baseline.py e 0002_token_version.py e trecho da rota e do serviço lado a lado. Observação: o CRUD simples permanece nos routers (seção 6.4).

[RNF08] — Confiabilidade e desempenho da automação (catraca virtual)

- **Categoria:** Confiabilidade / Desempenho.

- **Descrição:** O estado da catraca é persistido no banco e só muda por transições válidas da máquina de estados (RN07 e RN09); cada ciclo de validação deve concluir em menos de 1 segundo e ser registrado com o tempo de ciclo.
- **Componente / processo afetado:** `app/services/turnstile.py`, tabelas `turnstiles` e `access_logs`.
- **Condições iniciais:** Banco com 2 catracas e ao menos um aluno com aula agendada ou o simulador.
- **Procedimento de verificação:** Executar os cenários do simulador e a sequência de check-ins; conferir o campo `cycle_ms` dos eventos e a tentativa de transição inválida (`InvalidTransition`).
- **Métrica / parâmetro:** Tempo de ciclo < 1 s; nenhuma transição fora do mapa `TRANSITIONS`.
- **Resultado esperado:** Ciclos registrados com tempo; estados coerentes após reinício da API.
- **Evidência a coletar:** Testes test_checkin_*, test_bloqueio_apos_negativas_seguidas_e_rearme, test_simulador_cenarios e test_rnf08_ciclo_da_catraca_dentro_de_1_segundo; saída de docs/evidencias/evidencias-api.txt (24/10/2026), com ciclos de 0 a 1 ms; e o cartão “Tempo médio de ciclo” da tela Catraca.

2.3 Regras de Negócio (RN)

As Regras de Negócio definem as restrições lógicas e os processos que o software deve obedecer obrigatoriamente. São 16 regras do TrainUp, revisadas na Sprint 2 conforme o comportamento implementado:

- **RN01 — Vínculo Aluno–Personal:** Um Aluno está vinculado a no máximo um Personal Trainer por vez. O vínculo nasce de uma solicitação por código de convite que o Personal precisa aprovar (ou do cadastro do aluno feito pelo próprio Personal). Prioridade: Alta.
- **RN02 — Controle de acesso por perfil:** Um Aluno só acessa os próprios dados; um Personal só acessa alunos vinculados a ele; ações de gestão são exclusivas do Personal e a execução de treinos e o check-in são exclusivos do Aluno. Prioridade: Crítica.
- **RN03 — Conflito e validade de agendamento:** Não é permitido haver aulas sobrepostas (status AGENDADO ou CHECKIN) para o mesmo Personal; o horário não pode estar no passado (tolerância de 5 minutos) e a duração deve ser de 15 a 240 minutos. Aulas canceladas ou concluídas não bloqueiam horário. Prioridade: Alta.
- **RN04 — Ciclo de vida da aula:** As transições permitidas são AGENDADO → CHECKIN ou CANCELADO e CHECKIN → CONCLUIDO; CONCLUIDO e CANCELADO são finais. O status CHECKIN só é alcançado pela catraca (check-in). Prioridade: Alta.
- **RN05 — Participantes da aula:** Somente o aluno e o personal da aula podem consultá-la ou alterá-la; o Aluno só pode cancelar; o Personal pode cancelar e concluir. Prioridade: Alta.
- **RN06 — Janela de check-in e anti-fraude:** O check-in só é aceito para aula AGENDADO do próprio aluno, a partir de 15 minutos antes do início até o fim da aula; há um único check-in por aula (nova leitura é negada). Prioridade: Alta.

- **RN07 — Bloqueio da catraca:** Após 3 negativas seguidas a catraca entra em BLOQUEIO, ignora leituras e aciona o alarme por 60 segundos ou até o rearme manual do Personal; uma liberação zera as negativas. Prioridade: Alta.
- **RN08 — Limite de tentativas de login:** Após 5 falhas em 5 minutos para o mesmo IP + e-mail, novas tentativas dessa combinação são recusadas (429); há também um teto de 20 falhas por e-mail no mesmo intervalo, independentemente do IP. O IP considerado é o da conexão; o cabeçalho X-Real-IP só é aceito quando a API está atrás do proxy confiável (nginx), de modo que forjá-lo não burla o limite. Prioridade: Média.
- **RN09 — Máquina de estados e tempo de passagem:** A catraca só executa as transições válidas (TRAVADA → VALIDANDO → LIBERADA → AGUARDANDO_PASSAGEM → TRAVADA; TRAVADA → BLOQUEIO → TRAVADA); o estado é persistido no banco. Após a liberação há 10 segundos para a passagem, depois a trava religa. Prioridade: Alta.
- **RN10 — Cadastro, e-mail e senha:** O cadastro de Personal exige o código de cadastro do ambiente (sem ele, desabilitado). O e-mail é único no sistema e a senha deve ter ao menos 8 caracteres (máximo 72 bytes), com letras e números, sempre armazenada em hash bcrypt. Prioridade: Alta.
- **RN11 — Catálogo de exercícios:** Exercícios sem dono formam o catálogo base (somente leitura, 32 exercícios); cada Personal edita apenas os seus; o nome é único na biblioteca visível no momento da criação (a edição não revalida duplicidade); fichas só aceitam exercícios do catálogo base ou do Personal; exercício em uso não pode ser excluído. Prioridade: Média.
- **RN12 — Validação das séries:** Repetições de 1 a 200, carga de 0 a 1000 kg e número da série maior ou igual a 1; uma sessão finalizada não aceita novas séries. Prioridade: Alta.
- **RN13 — Recorde e estagnação:** Há recorde quando a carga máxima da sessão supera a maior carga anterior do aluno no exercício. Há estagnação quando o exercício tem ao menos 3 sessões e a carga máxima não sobe há 2 ou mais sessões (limite configurável, STAGNATION_N, nos alertas e no painel; o relatório usa critério fixo equivalente ao padrão). Nas análises, “sessão” de um exercício é o dia (America/Sao_Paulo) em que houve séries dele. Prioridade: Alta.
- **RN14 — Dicas educativas:** As dicas de nutrição e recuperação são conteúdo educativo geral, sem prescrição nem indicação de suplementos, sempre com aviso para procurar nutricionista ou médico. Prioridade: Média.
- **RN15 — Fuso horário:** Todas as datas são gravadas em UTC; a interface e os relatórios usam America/Sao_Paulo (agrupamento por dia, períodos e logs). Prioridade: Média.
- **RN16 — Transparência da simulação:** Eventos gerados pelo simulador são marcados como simulados (`simulated = true`, com o nome do cenário) e a tela exibe a identificação “Simulação”; a catraca nunca é apresentada como equipamento real. Prioridade: Média.

2.4 Rastreabilidade: requisito, implementação e teste

A tabela relaciona cada requisito funcional ao caso de uso, aos endpoints, à tela do front-end e aos testes automatizados que o verificam. Todos os requisitos funcionais possuem ao menos um teste automatizado.

RF	Caso de uso	Endpoint(s)	Tela	Teste(s)
RF01	Registrar-se	`POST /auth/register`	Login (cadastro)	test_cadastro_de_personal_exige_codigo, test_senha_fraca_e_rejeitada
RF02	Autenticar-se	`POST /auth/login`, `GET /auth/me`	Login	test_rate_limit_do_login, test_rate_limit_por_email_ignora_ip_forjado, test_x_real_ip_ignorado_sem_proxy_confiavel; API: 401
RF03	Recuperar senha	`POST /auth/forgot`, `/auth/reset`	Login (recuperação)	test_recuperar_senha_token_invalido_e_senha_fraca, test_troca_de_senha_derruba_sessoes_antigas
RF04	Solicitar vínculo	`POST /links/request`, `GET /links/mine`	Aluno / Início	test_fluxo_de_vinculo_com_aprovacao
RF05	Aprovar/recusar vínculo	`GET /links/pending`, `POST /links/{id}/respond`	Personal / Alunos	test_fluxo_de_vinculo_com_aprovacao, test_recusar_nao_vincula
RF06	Cadastrar aluno	`POST /users`	Personal / Alunos	Uso em todos os testes (make_aluno)
RF07	Consultar/editar/remover aluno	`GET/PUT/DELETE /users/{id}`	Personal / Alunos, Perfil do aluno	test_listagem_de_usuarios_por_perfil, test_atualizar_usuario_nome_objetivo_e_senha, test_usuarios_de_outro_personal_sao_invisiveis, test_remover_aluno_apaga_historico_completo
RF08	Gerenciar exercícios	`/exercises`	Personal / Exercícios	test_aluno_nao_cria_exercicio_nem_edita_catalogo_base, test_biblioteca_propria_de_exercicios
RF09	Gerenciar fichas	`/workouts`	Personal / Treinos	test_ficha_de_treino_crud_e_ficha_do_dia, test_ficha_so_para_aluno_vinculado
RF10	Consultar fichas	`GET /workouts`, `/workouts/today`	Aluno / Treinar	test_ficha_de_treino_crud_e_ficha_do_dia
RF11	Executar sessão	`POST /sessions`, `/sessions/{id}/sets`	Aluno / Treinar	test_sessao_de_treino_resumo_com_recorde, test_serie_em_sessao_finalizada_e_rejeitada, test_sessao_fluxo_completo_listagem_e_resumo, test_sessao_isolamento_e_erros, test_registro_avulso_valida_entrada_e_perfil
RF12	Finalizar e resumir	`POST /sessions/{id}/finish`	Aluno / Treinar	test_sessao_de_treino_resumo_com_recorde
RF13	Sessões e evolução	`GET /analytics/evolution`, `/sessions`	Aluno / Evolução	test_sessao_fluxo_completo_listagem_e_resumo, test_evolucao_calcula_media_e_progressao, test_evolucao_sem_dados_e_sem_aluno_id, test_personal_nao_acessa_aluno_de_outro
RF14	Alertas de estagnação	`GET /analytics/stagnation`	Visão geral, Alunos	test_streak_de_estagnacao, test_alerta_de_estagnacao
RF15	Dicas	`GET /analytics/tips`	Aluno / Evolução	test_dicas_por_objetivo_com_aviso
RF16	Relatório	`GET /reports/aluno/{id}`	Personal / Relatórios, Aluno / Evolução	test_relatorio_json_e_pdf, test_aluno_so_ve_o_proprio_relatorio
RF17	Observação do personal	`PUT /reports/aluno/{id}/note`	Personal / Relatórios	test_observacao_do_personal_no_relatorio
RF18	Relatório em PDF	`GET /reports/aluno/{id}/pdf`	Relatórios / Evolução	test_relatorio_json_e_pdf
RF19	Agendar aula	`POST /schedules`	Agenda	test_conflito_de_agenda
RF20	Agenda e cancelamento	`GET /schedules`, `PATCH /schedules/{id}/status`	Agenda	test_transicoes_de_status
RF21	Concluir aula	`PATCH /schedules/{id}/status`	Agenda	test_checkin_leva_a_concluido

RF	Caso de uso	Endpoint(s)	Tela	Teste(s)
RF22	Check-in na catraca	`POST /automacao/checkin`	Início, Agenda	test_checkin_dentro_da_janela_libera_a_catraca, test_segundo_checkin_da_mesma_aula_e_negado, test_fora_da_janela_e_negado
RF23	Confirmar passagem	`POST /automacao/passagem`	Início e Agenda (aluno); Catraca (personal)	test_passagem_so_pelo_aluno_liberado_ou_seu_personal, test_simulador_cenarios
RF24	Estado e histórico	`GET /automacao/status`, `/logs`, `/turnstiles`	Catraca; cartão de check-in (Início/Agenda)	test_simulador_cenarios (estatísticas), test_status_estatisticas_e_catracas, test_logs_filtros_e_privacidade
RF25	Simular cenários	`POST /automacao/simulate`	Catraca	test_simulador_cenarios, test_aluno_nao_usa_simulador
RF26	Rearmar catraca	`POST /automacao/reset`	Catraca	test_bloqueio_apos_negativas_seguidas_e_rearme
RF27	Painel do personal	`GET /dashboard/personal`	Personal / Visão geral	test_painel_do_personal
RF28	Encerrar sessão	POST /auth/logout	Layout / menu do usuário	test_logout_revoga_o_token; test_troca_de_senha_derruba_sessoes_antigas

3 Cronograma do Projeto

O projeto segue o calendário do Projeto Integrador Interdisciplinar II (2º semestre de 2026), organizado em três Sprints com entregas e demonstrações, da abertura em 03/08/2026 até a Mostra de Projetos (07 a 11/12/2026) e a recuperação (14/12/2026). A Sprint 1 concentrou o levantamento de requisitos, a modelagem e o protótipo; a Sprint 2 converteu essa especificação em um fluxo principal integrado e executável; a Sprint 3 está reservada para a conclusão, os testes finais, a implantação e a preparação da Mostra. Abaixo estão o controle de entregáveis da Sprint 1 (encerrada) e o da Sprint 2, com responsável, prazo e situação de cada item.

3.1 Planejamento de Sprints

A execução segue a metodologia ágil **Scrum**, em ciclos de aproximadamente **5 semanas** (Sprint 1: 03/08 a 14/09; Sprint 2: 21/09 a 26/10; Sprint 3: 09/11 a 30/11/2026), com encontros presenciais semanais e períodos de desenvolvimento autônomo entre eles.

- **Sprint 01 (concluída):** levantamento de requisitos, modelagem de dados e do domínio, diagramas UML, protótipo e repositório.
- **Sprint 02 (concluída):** implementação do fluxo principal de ponta a ponta com banco integrado (Alembic e seed), interface responsiva conectada à API, validações, testes iniciais com evidências, documentação Swagger e atualização da ERS.
- **Sprint 03 (planejada):** conclusão das funcionalidades obrigatórias, testes finais e de integração, documentação do ambiente de desenvolvimento em nuvem (EC2 com code-server), manual de instalação, matriz de rastreabilidade final e Mostra de Projetos.

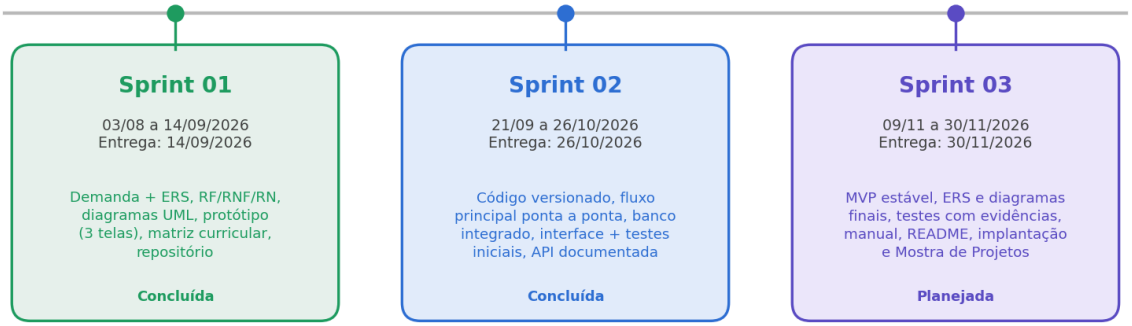


Figura 2 – Linha do tempo das Sprints do TrainUp.

3.2 Controle de entregáveis — Sprint 1 (encerrada)

ID	Entregável	Referência no Guia	Data sugerida	Responsável	Status	%	Prazo final
1	Documento inicial da demanda (com evidência SAGA ativa)	Seção 2 (Entregável 1)	10/08/2026	Todos	Concluído	100%	14/09/2026
2	ERS — Especificação de Requisitos de Software (consolidação final)	Seção 3 (Entregável 2)	14/09/2026	Bárbara	Concluído	100%	14/09/2026
3	Lista priorizada de RF, RNF e RN	Seção 4 (Entregável 3)	17/08/2026	Bárbara	Concluído	100%	14/09/2026
4	Especificação textual dos fluxos, subfluxos e exceções dos RFs	Seção 5 (Entregável 4)	24/08/2026	Luana	Concluído	100%	14/09/2026

ID	Entregável	Referência no Guia	Data sugerida	Responsável	Status	%	Prazo final
5	Cenários textuais de verificação dos RNFs	Seção 6 (Entregável 5)	24/08/2026	Ruan	Concluído	100%	14/09/2026
6	Diagrama de Casos de Uso	Seção 7 (Entregável 6)	31/08/2026	Ruan	Concluído	100%	14/09/2026
7	Diagrama de Classes	Seção 8 (Entregável 7)	31/08/2026	Pedro	Concluído	100%	14/09/2026
8	Diagrama Entidade-Relacionamento (DER) + dicionário de dados	Seção 9 (Entregável 8)	31/08/2026	Pedro	Concluído	100%	14/09/2026
9	Dois diagramas de sequência dos processos principais	Seção 10 (Entregável 9)	31/08/2026	Luana	Concluído	100%	14/09/2026
10	Protótipo das 3 principais interfaces e navegação	Seção 11 (Entregável 10)	31/08/2026	Ana	Concluído	100%	14/09/2026
11	Matriz de integração das unidades curriculares	Seção 12 (Entregável 11)	17/08/2026	Ana	Concluído	100%	14/09/2026
12	Repositório com README e estrutura inicial organizada	Seção 13 (Entregável 12)	03/08/2026	Pedro	Concluído	100%	14/09/2026

3.3 Controle de entregáveis — Sprint 2 (prazo final: 26/10/2026)

Situação em 25/10/2026:

ID	Entregável	Referência no Guia	Data sugerida	Responsável	Apoio	Status	%	Prazo final
1	Código-fonte atualizado e versionado (correções da Sprint 1 incorporadas)	Seção 2 (Entregável 1)	21/09/2026	Pedro		Concluído	100%	26/10/2026
2	Fluxo principal funcional de ponta a ponta (cadastro de aluno → treino → evolução → indicadores → relatório)	Seção 3 (Entregável 2)	28/09/2026	Ana		Concluído	100%	26/10/2026
3	Banco de dados criado e integrado (migrations Alembic + seed de dados)	Seção 4 (Entregável 3)	21/09/2026	Pedro		Concluído	100%	26/10/2026
4	Interface responsiva integrada ao back-end (telas React consumindo a API real)	Seção 5 (Entregável 4)	28/09/2026	Pedro		Concluído	100%	26/10/2026
5	Validações e tratamento básico de erros (formulários, códigos HTTP, mensagens)	Seção 6 (Entregável 5)	18/10/2026	Ana		Concluído	100%	26/10/2026
6	Testes iniciais com evidências (unitários e manuais, prints do Swagger em docs/api/)	Seção 7 (Entregável 6)	18/10/2026	Ruan		Concluído	100%	26/10/2026
7	Documentação da API (Swagger/OpenAPI revisado e exportado; prints e openapi.json em docs/api/)	Seção 8 (Entregável 7)	19/10/2026	Pedro	Ruan	Concluído	100%	26/10/2026
8	ERS, diagramas e backlog atualizados (refletindo decisões reais da implementação; backlog em docs/BACKLOG.md)	Seção 9 (Entregável 8)	19/10/2026	Bárbara		Concluído	100%	26/10/2026
9	Registro de dificuldades, decisões e mudanças de escopo (atas + CHANGELOG.md)	Seção 10 (Entregável 9)	26/10/2026	Luana		Concluído	100%	26/10/2026

Os entregáveis da Sprint 3 (MVP funcional e estável, ERS final, diagramas atualizados, relatório de testes, matriz de rastreabilidade final, manual de instalação, README completo, backlog final, limitações e evolução, apresentação da Mostra e evidências de participação individual) serão detalhados no planejamento de 09/11/2026.

4 Protótipo do produto

Nesta seção apresentam-se as interfaces do sistema implementado. O protótipo de alta fidelidade da Sprint 1 (tema escuro) foi substituído pela interface real em React + Tailwind CSS, que adota identidade visual clara com menu lateral azul-marinho, cores com contraste AA e navegação adaptada ao celular (barra inferior). As telas abaixo foram capturadas do sistema em execução com os dados de demonstração (pasta `docs/evidencias/prints/`).

Principais telas:

Tela 1 — Login (/login): cartão com e-mail e senha, links “Esqueci minha senha” e “Cadastre-se” (cadastro, recuperação e redefinição de senha no mesmo componente). Erros de credencial aparecem em faixa vermelha acima do botão.

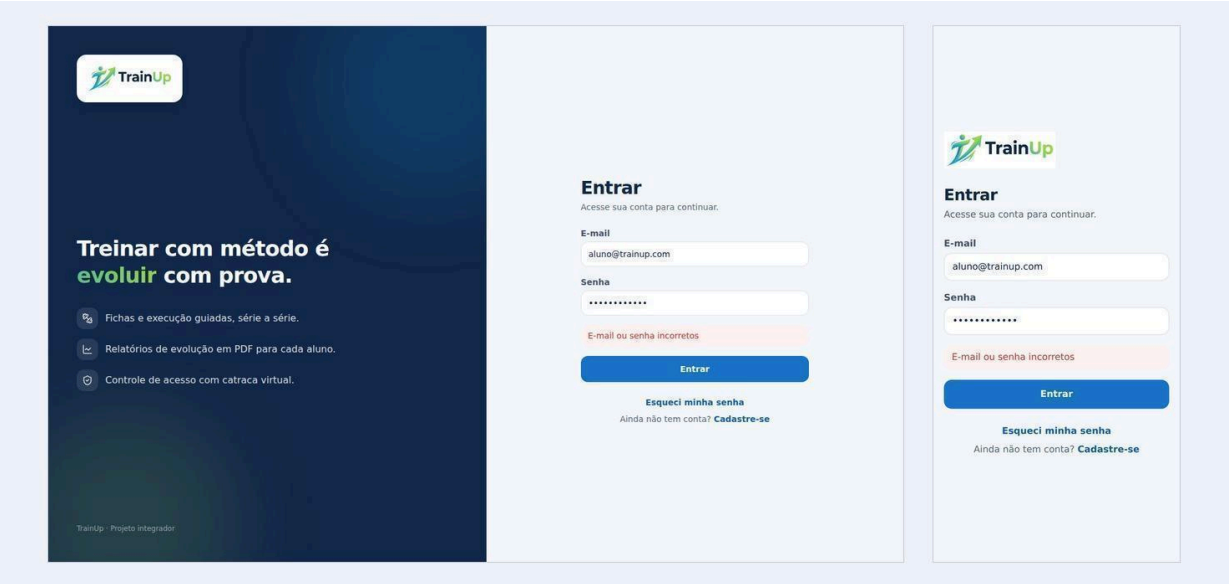


Figura 3 – Tela de login com erro de credencial (desktop 1440 px e celular 375 px).

Tela 2 — Visão geral do Personal (/personal): aviso de solicitações de vínculo, cartões de alunos ativos, aulas do dia e alertas de estagnação, lista de próximas aulas e alertas por aluno/exercício.

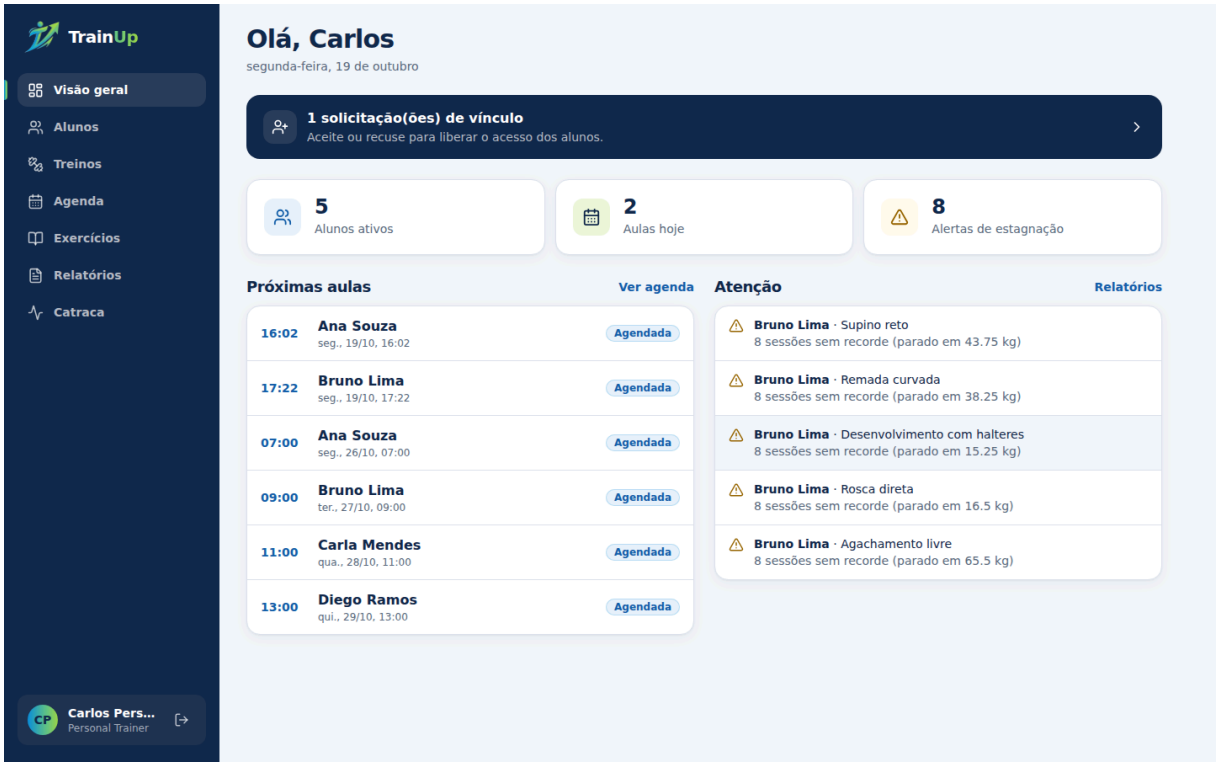


Figura 4 – Visão geral do Personal Trainer (desktop).

Tela 3 — Treinar (Aluno) (/aluno/treinar): fichas do dia com exercícios e séries e botão “Iniciar este treino”; durante o treino, registro de repetições e carga série a série com timer de descanso e, ao finalizar, resumo com volume, comparação e recordes.

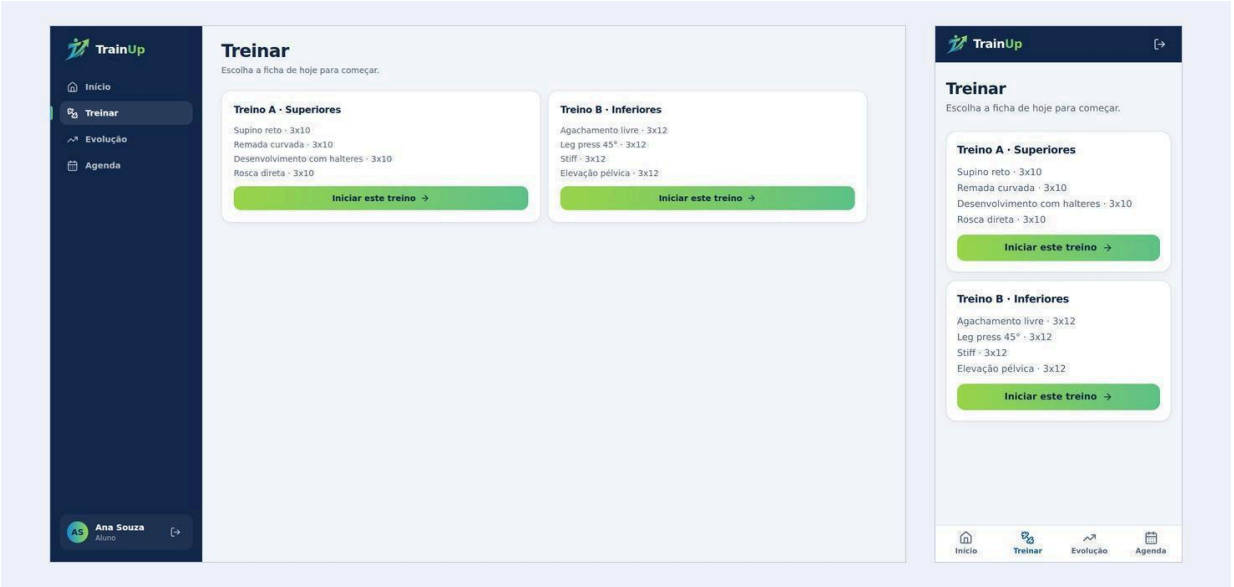


Figura 5 – Tela Treinar do Aluno (desktop e celular).

Tela 4 — Evolução (Aluno) (/aluno/evolucao): seleção de período (7 dias, 30 dias, 3 meses, personalizado), cartões de sessões, presença e recordes, resumo textual automático, gráfico de evolução de carga por exercício e botão “Baixar PDF”.



Figura 6 – Evolução do Aluno (parte superior da tela; o gráfico de carga fica abaixo).

Tela 5 — Catraca (Personal) (/automacao): supervisório da catraca virtual com semáforo de estado, entradas e saídas do CLP, estatísticas do dia, simulador de sensores com cinco cenários, histórico e lógica de controle, organizados nas abas Supervisório, Histórico e Lógica de controle. A tela é rotulada “Simulação”.



Figura 7 – Painel “Catraca” (automação da catraca virtual) do Personal.

Fluxo de navegação entre as telas:

- Login com credenciais de Personal → Visão geral (/personal) → menu lateral: Alunos, Treinos, Agenda, Exercícios, Relatórios e Catraca. No celular, a barra inferior mostra Visão geral, Alunos, Treinos e Agenda; Exercícios, Relatórios, Catraca e Sair ficam em “Mais”.
- Login com credenciais de Aluno → Início (/aluno) → barra inferior/menu lateral: Treinar, Evolução e Agenda.
- Aluno: Início → Treinar → iniciar ficha → registrar séries → finalizar → resumo da sessão → Evolução → Baixar PDF.

- Aluno: Agenda → agendar aula → “Fazer check-in” (janela de 15 min antes) → catraca liberada; Personal acompanha em Catraca → Histórico.

5 Diagramas UML

5.1 Diagrama de Casos de Uso

O diagrama representa os atores do TrainUp e os casos de uso do sistema implementado. O ator generalizado Usuário reúne o que Personal Trainer e Aluno têm em comum: registrar-se, autenticar-se, recuperar senha, encerrar sessão, agendar aula, consultar agenda, cancelar aula, consultar fichas de treino, consultar o relatório de evolução e consultar o estado da catraca. O Personal Trainer cadastra, consulta e remove alunos, responde solicitações de vínculo, gerencia exercícios e fichas de treino, consulta o painel, a evolução de carga dos alunos, os alertas de estagnação e o histórico de acessos, registra observação no relatório, conclui aulas e opera a catraca (simular cenário e rearmar). O Aluno solicita vínculo, executa a sessão de treino, realiza check-in, confirma a passagem na catraca e consulta dicas educativas. O Personal também pode registrar a passagem do aluno liberado (RF23); no diagrama, o caso de uso “Confirmar passagem na catraca” aparece associado apenas ao Aluno.

Destacam-se: «include» de “Registrar séries” e “Finalizar sessão e obter resumo” em “Executar sessão de treino” (a sessão sempre registra séries e termina com o resumo); e «extend» de “Exportar relatório em PDF” sobre “Consultar relatório de evolução” (o PDF é opcional). As regras de acesso — o Aluno acessa apenas os próprios dados e o Personal apenas alunos vinculados (RN02), e o agendamento não admite sobreposição de horário do mesmo personal (RN03) — são aplicadas pela API e estão descritas nos requisitos.

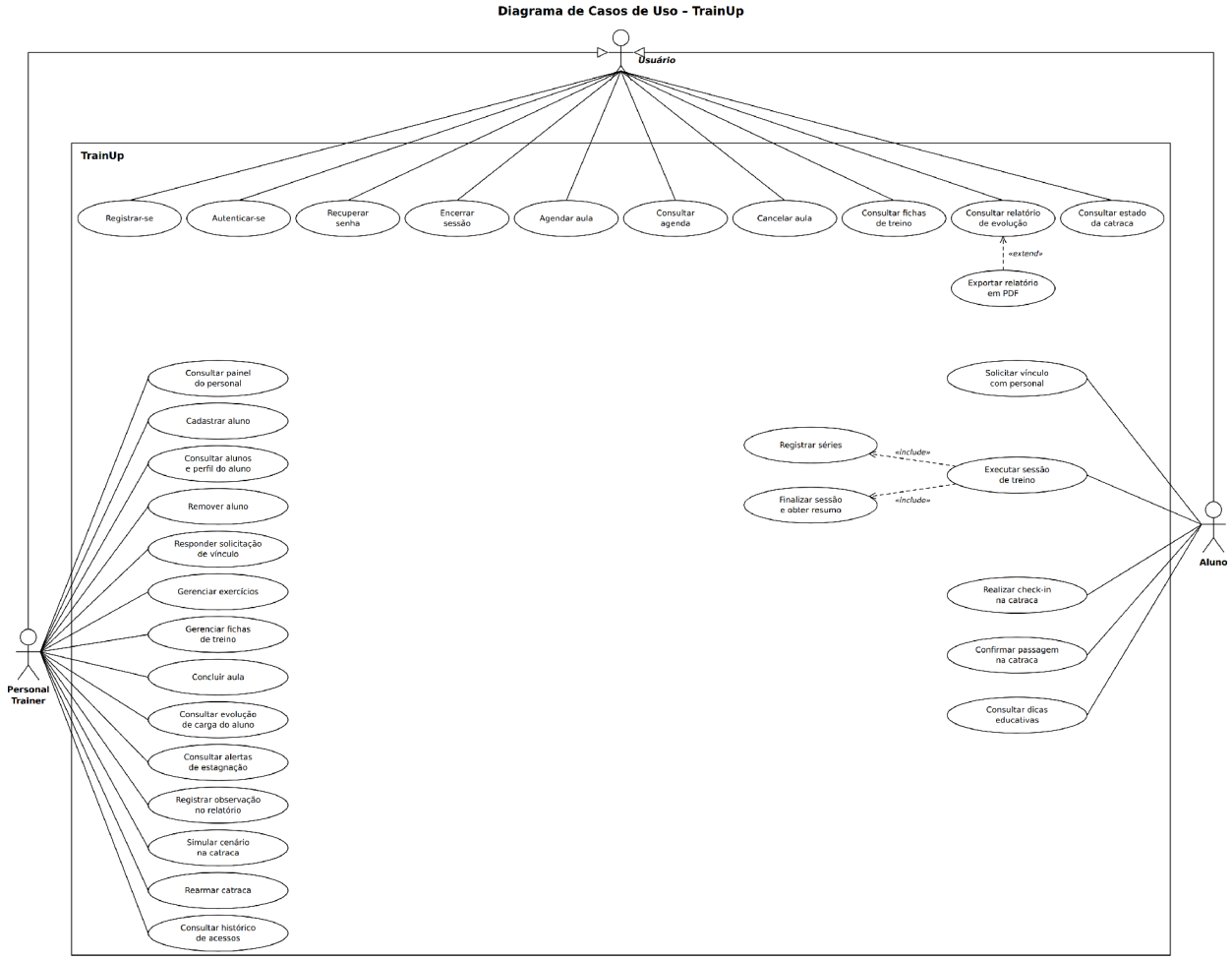


Figura 8 – Diagrama de Casos de Uso — TrainUp.

5.2 Diagrama de Classes

O diagrama de classes representa o domínio persistido do TrainUp (11 classes) e cinco enumerações (Role, StatusVinculo, StatusAgendamento, EstadoCatraca e EventoAcesso). User concentra os dois perfis por meio do atributo role e se auto-relaciona (um personal “orienta” vários alunos, RN01). O personal e o aluno se associam a Workout (ficha), composta por WorkoutItem (1..*), que referencia Exercise; exercício com owner nulo pertence ao catálogo base (somente leitura, RN11). WorkoutSession agrupa os registros Progress (séries). Schedule representa a aula e seu ciclo de status; LinkRequest representa a solicitação de vínculo; ReportNote guarda a observação do personal. Turnstile e AccessLog modelam a catraca e o histórico; as transições do estado seguem a máquina de estados do serviço TurnstileFSM, que não é uma classe persistida (RN09). O atributo is_base de Exercise é derivado (owner nulo). As referências personal_id/schedule_id do AccessLog e last_aluno_id da catraca são colunas inteiras sem chave estrangeira, por decisão de projeto (seção 6.2).

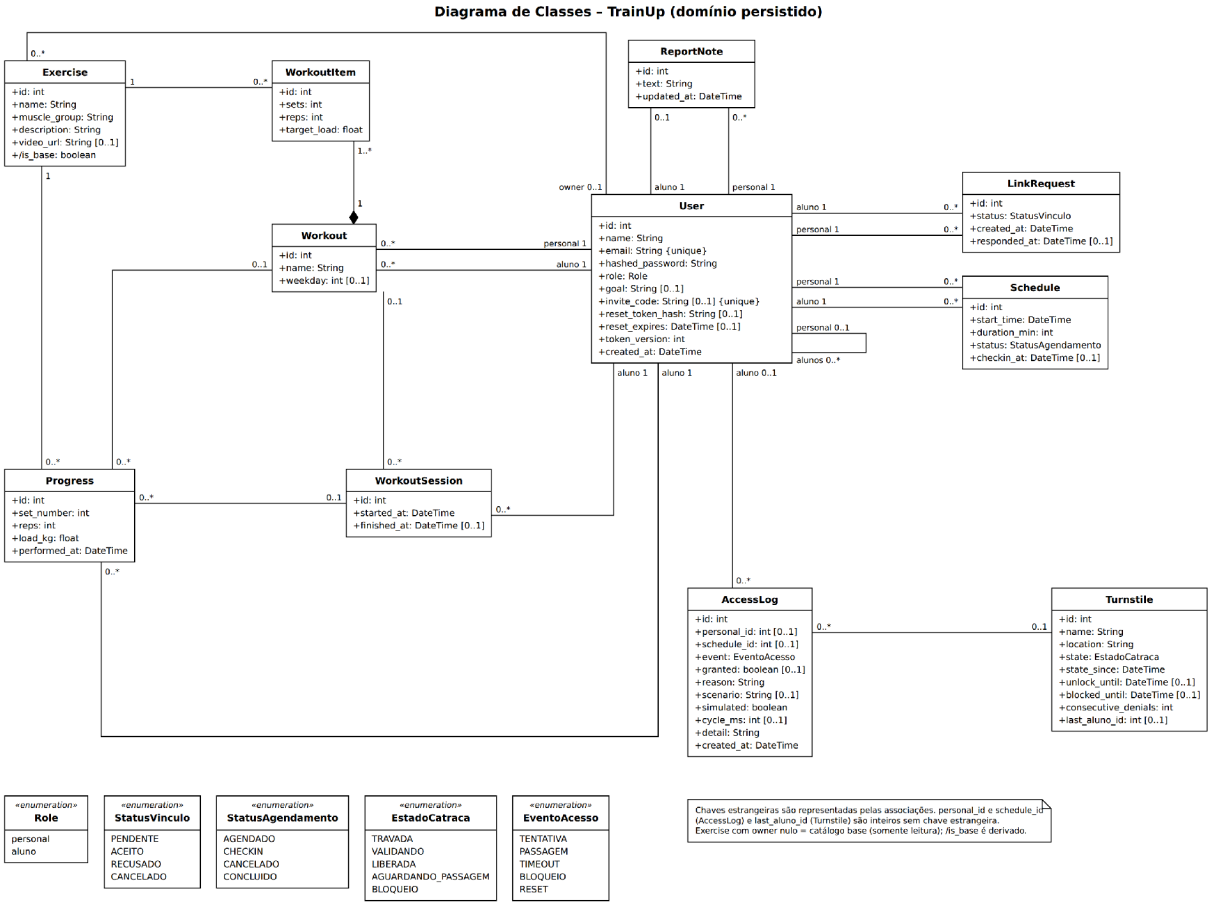


Figura 9 – Diagrama de Classes — TrainUp (domínio persistido).

5.3 Diagrama Entidade-Relacionamento (DER)

O banco de dados PostgreSQL do TrainUp é composto por 11 tabelas: `users`, `link_requests`, `exercises`, `workouts`, `workout_items`, `workout_sessions`, `progress`, `schedules`, `report_notes`, `turnstiles` e `access_logs`. A tabela `users` guarda os dois perfis e possui auto-relacionamento (`personal_id`, vínculo ativo, 1:N, RN01). Fichas, sessões, séries, aulas e solicitações de vínculo referenciam `users` pelo aluno e/ou pelo personal; `workout_items` liga ficha e exercício; `progress` liga série, exercício, ficha e sessão. `report_notes` é única por aluno. A cardinalidade mínima de 1 item por ficha (workout_items por workouts) é regra da API, não do banco. As chaves estrangeiras são aplicadas pelo banco, inclusive no SQLite dos testes (seção 6.2, D13). O dicionário de dados completo está no Apêndice A.

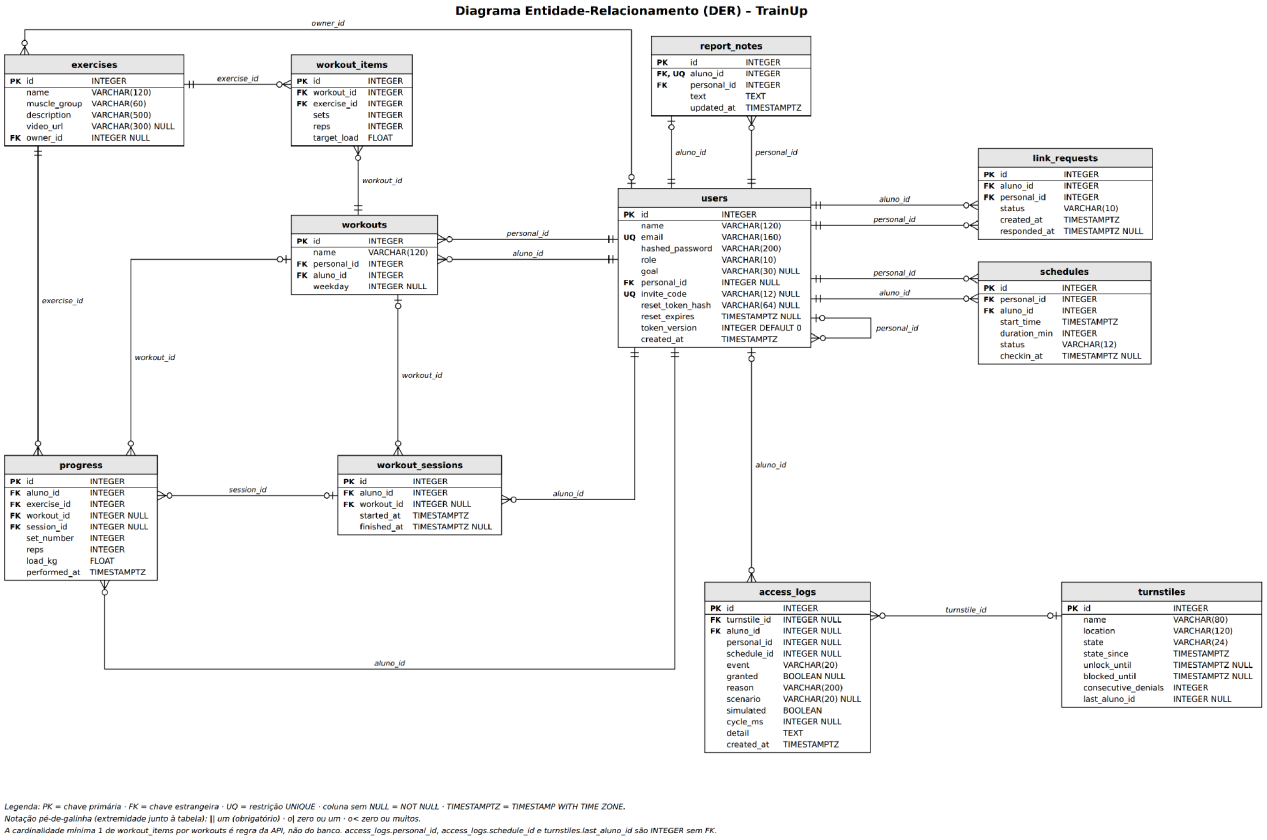


Figura 10 – Diagrama Entidade-Relacionamento — TrainUp.

5.4 Diagramas de Sequência

5.4.1 Diagrama de Sequência — Executar sessão de treino [RF10 a RF12]

Fluxo do Aluno em quatro etapas. (1) Selecionar a ficha do dia: o front-end chama GET /workouts/today?weekday=n, que devolve as fichas do aluno autenticado para o dia da semana (ou as sem dia definido). (2) Iniciar a sessão: POST /sessions cria a sessão (404 se a ficha não for do aluno). (3) Registrar séries (loop): para cada série concluída, POST /sessions/{id}/sets valida a sessão (404/409) e o exercício e insere em progress; limites de validação resultam em 422. (4) Finalizar: POST /sessions/{id}/finish grava finished_at e o serviço session_summary calcula volume, recordes (carga máxima maior que a anterior) e variação em relação à sessão anterior, devolvendo o resumo. Todas as chamadas levam o JWT (401 sem token; 403 para perfil diferente de aluno).

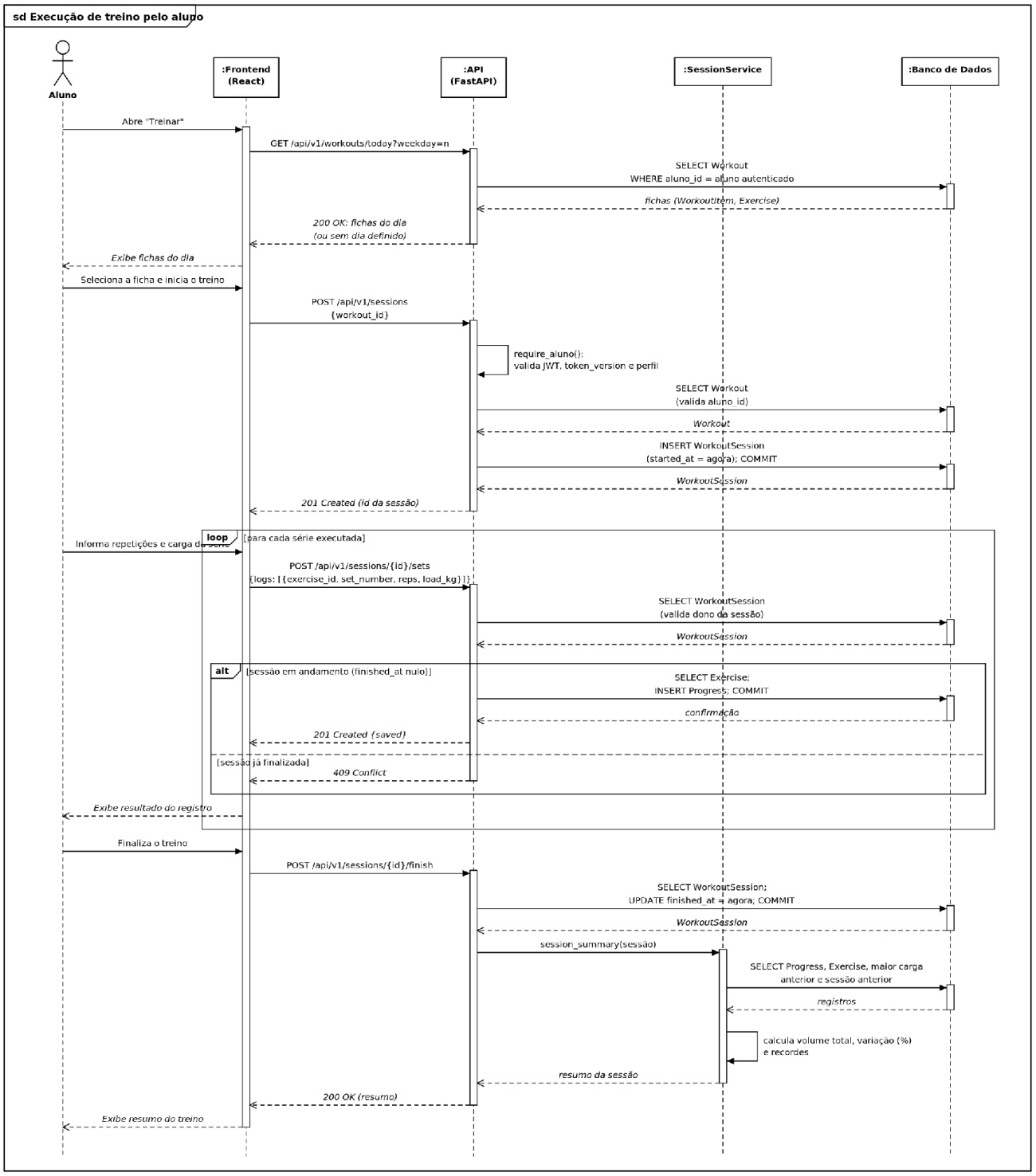


Figura 11 – Diagrama de Sequência 1 — Executar sessão de treino (Aluno).

5.4.2 Diagrama de Sequência — Realizar check-in na catraca virtual [RF22]

O Aluno aciona “Check-in” na aula agendada e o front-end chama POST /automacao/checkin com o schedule_id e o JWT. A API valida o acesso (require_aluno: JWT, versão do token e perfil), atualiza o estado da catraca por tempo (get_turnstile/refresh), trava a linha da catraca (SELECT ... FOR UPDATE, no PostgreSQL) e carrega a aula; o agendamento (Booking) só é criado se a aula pertencer ao aluno autenticado. Em seguida chama run_cycle no Turnstile Service: em BLOQUEIO a leitura é ignorada; fora dele, a máquina de estados vai de TRAVADA a VALIDANDO e a validação

confere o status AGENDADO e a janela de check-in (15 minutos antes do início até o fim). Se permitido: VALIDANDO → LIBERADA → AGUARDANDO_PASSAGEM por 10 s, negativas zeradas e a aula vai a CHECKIN, com checkin_at. Se negado: VALIDANDO → TRAVADA e contagem de negativas; ao atingir 3, a catraca vai a BLOQUEIO por 60 s. O ciclo grava o AccessLog e confirma a transação (catraca, aula e log); a resposta traz granted, reason, event, log, transitions e o estado da catraca. O agendamento da aula (RF19) é feito antes, por POST /schedules.

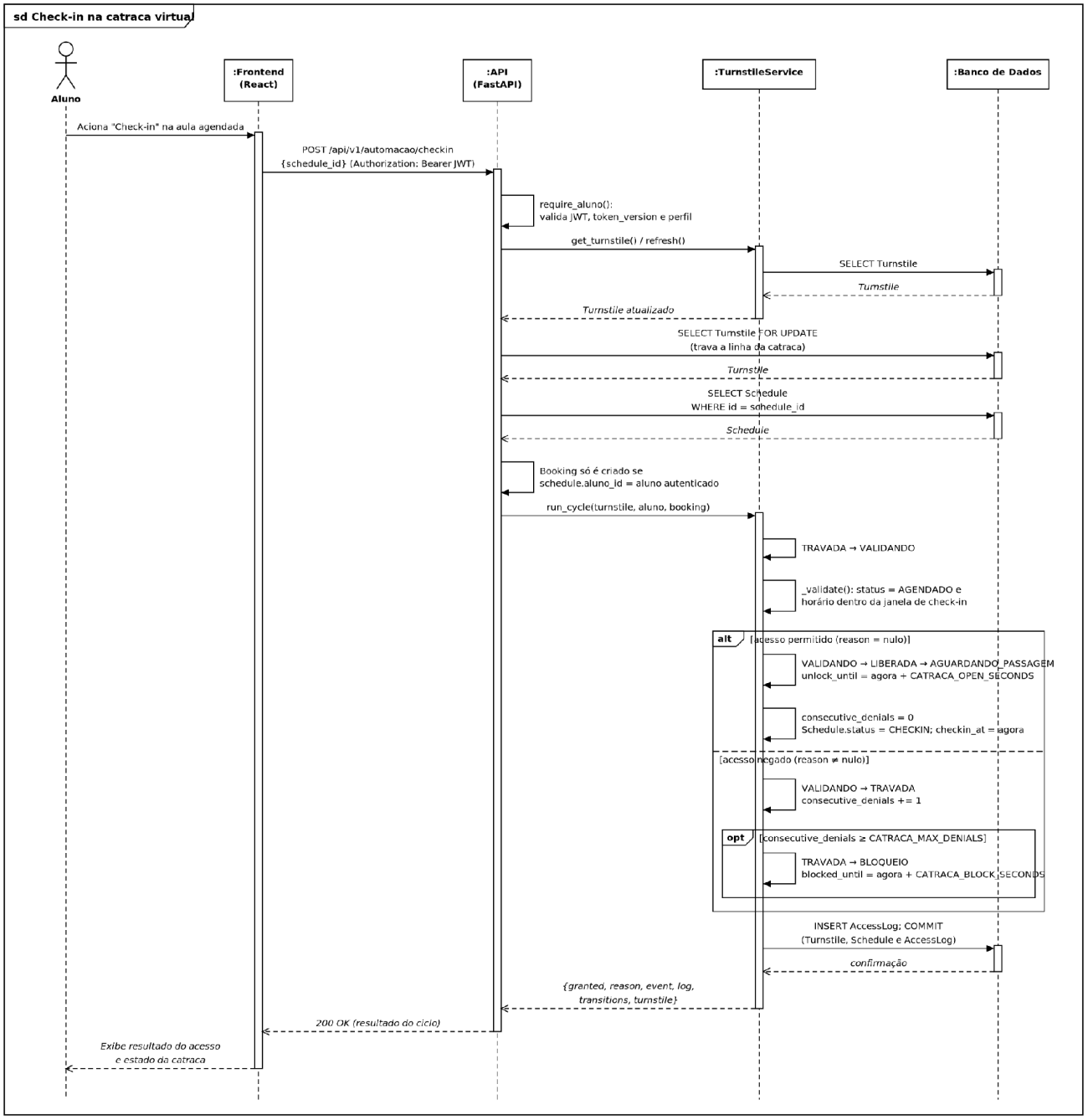


Figura 12 – Diagrama de Sequência 2 — Realizar check-in na catraca virtual (Aluno).

6 Decisões de Projeto e Mudanças de Escopo

Esta seção registra o que mudou entre a especificação da Sprint 1 e o sistema implementado na Sprint 2, e por quê. Ela atende ao registro de decisões e mudanças de escopo da Sprint 2 e justifica as diferenças entre esta ERS (versão 2.2) e a versão 1.0. As decisões foram tomadas durante a Sprint 2 (21/09 a 26/10/2026).

6.1 Mudanças de escopo (Sprint 1 → Sprint 2)

Item	Sprint 1 (planejado)	Sprint 2 (implementado)	Motivo
Núcleo do MVP	Evolução física registrada manualmente pelo Personal (peso, altura, medidas, carga de referência, desempenho 0–10), imutável, com origem manual/simulado (RF08–RF10).	Evolução calculada das séries que o Aluno executa em sessões de treino (carga, repetições, volume, recordes, estagnação). Sem peso/medidas corporais.	Gera dados reais pelo uso, elimina o retrabalho do personal e reforça Ciência de Dados. RN04/RN05 antigas substituídas por RN12–RN13.
Relatórios	Relatório persistido com snapshot JSON do período e resumo textual.	Relatório calculado sob demanda, com frequência às aulas, volume semanal, recordes, observação do personal e PDF (reportlab).	Evita dados duplicados e desatualizados; o PDF atende à necessidade de compartilhar com o aluno.
Modelo de usuário	`Usuario` + perfis `PersonalTrainer` e `Aluno` (1:1), chaves UUID.	Tabela única `users` com `role` (personal/aluno), chaves inteiras e vínculo por `personal_id` (auto-relacionamento).	Simplifica consultas e permissões; reduz tabelas no MVP.
Vínculo aluno–personal	Aluno escolhia o personal em lista pública (`GET /api/personal-trainers`) e o vínculo era imediato.	Código de convite de 8 caracteres, solicitação PENDENTE e aprovação do personal (`link_requests`); lista pública removida.	Privacidade e controle do personal sobre quem acessa seus dados (RN01, RN02).
Cadastro de Personal	Aberto a qualquer visitante.	Exige código de cadastro do ambiente; sem o código, o cadastro público de personal fica desabilitado.	Evita contas de personal falsas.
Agenda	Serviços, Disponibilidade e Agendamento PENDENTE → CONFIRMADO/REJEITADO, com confirmação do personal (RF12–RF15).	Aulas (`schedules`) AGENDADO → CHECKIN → CONCLUIDO ou CANCELADO; o aluno agenda para si e o personal para aluno vinculado; conflito considera aulas ativas. Serviços e disponibilidade removidos.	Integra a agenda ao check-in da catraca (automação) e corta escopo secundário.
Automação industrial	Pipeline de coleta/processamento de dados de treino com origem manual/simulada (conceito).	Catraca virtual com máquina de estados persistida, 2 catracas, entradas/saídas de CLP, simulador de 5 cenários, painel SCADA e histórico (`access_logs`). Novos RF22–RF26.	Evidencia a UC de Automação Industrial no contexto de academias, funcionando sem hardware.
Exercícios	Catálogo reutilizável; exercício criado automaticamente pelo nome na ficha.	Catálogo base somente leitura (32 exercícios) + biblioteca própria por personal; fichas podem ter dia da semana.	Evita duplicidades e vazamento de exercícios entre personais (RN11).
Fichas de treino	Treino com itens ordenados e observações.	Ficha com itens (séries, repetições, carga-alvo) e `weekday` opcional; “ficha do dia” para o aluno.	Alimenta a tela Treinar do aluno.
Painel (dashboard)	RF16, prioridade baixa (desejável).	Implementado como Visão geral do personal e Início do aluno (RF27).	Aproveita indicadores prontos e melhora a demonstração.
Autenticação	JWT com perfil_id, tipo, nome e e-mail; passlib.	JWT com sub, role e versão do token (480 min), bcrypt direto, rate limit por IP/e-mail e teto por conta, logout com revogação de tokens e recuperação de senha por token (30 min).	Segurança (RNF01); o token não carrega dados pessoais.
API e organização do código	Rotas /api/... com camadas service e repository.	API versionada em /api/v1, com routers, services, models e schemas; sem camada repository (consultas via SQLAlchemy nos routers/serviços).	Menos código intermediário no MVP; regras complexas mantidas em services (RNF07).
Banco de dados	UUID, NUMERIC, JSONB e ENUM nativo; 11 tabelas.	11 tabelas com chaves inteiras, FLOAT e VARCHAR para enumerações validadas em código; Alembic (0001 e 0002); SQLite em testes/dev local.	Portabilidade para testes e simplicidade; integridade garantida pelo código e pelos testes.

Item	Sprint 1 (planejado)	Sprint 2 (implementado)	Motivo
Interface	Protótipo em tema escuro.	Interface clara responsiva, menu lateral (desktop) e barra inferior (celular), contraste AA.	Usabilidade e acessibilidade (RNF04).

Mantido fora do escopo: pagamentos, chat, hardware real de catraca, aplicativo nativo, envio de e-mail e verificação de e-mail.

6.2 Decisões técnicas e de produto

ID	Decisão	Descrição
D01	Fuso horário	Datas gravadas em UTC; interface e relatórios em America/Sao_Paulo, com conversão explícita nos campos de data/hora (RN15).
D02	Suplementos removidos	Substituídos por “Dicas de nutrição e recuperação”, conteúdo educativo geral com aviso para procurar nutricionista (RN14). Opção mais segura.
D03	Estado da catraca no banco	Máquina de estados persistida e validada por mapa de transições; operações críticas usam bloqueio de linha (SELECT FOR UPDATE) no PostgreSQL para serializar leituras simultâneas. O sistema suporta várias catracas (RN09).
D04	Simulação identificada	Eventos do simulador têm `simulated = true`, nome do cenário e rótulo “Simulação” na tela (RN16). Em produção real, o check-in viria do leitor físico (RFID/QR).
D05	Referências sem chave estrangeira	`access_logs.personal_id`, `access_logs.schedule_id` e `turnstiles.last_aluno_id` são inteiros sem FK, para que o histórico de acessos sobreviva à remoção de aulas.
D06	Recuperação de senha sem e-mail	Token de 30 minutos (somente o hash é guardado) impresso no log do backend; envio de e-mail fica como evolução.
D07	Rate limit em memória	5 falhas em 5 minutos por IP + e-mail e teto de 20 falhas por e-mail, independentemente do IP; armazenamento em memória, adequado a um worker no MVP.
D08	Infraestrutura	Docker Compose (db, backend, frontend/nginx); somente a porta do nginx exposta (80 por padrão; 8081 na EC2); `SECRET_KEY` obrigatória em produção; Swagger em /api/docs somente com ENABLE_DOCS=true; o desenvolvimento ocorre em uma instância AWS EC2 com code-server (VS Code).
D09	Identidade visual	Azul #1A73C8 em botões e links (contraste AA); azul vivo e verde-limão apenas em gráficos e preenchimentos; logo recortada da imagem original, sem redesenho vetorial.
D10	Migrations	Alembic roda sozinho na subida da API; 0001_baseline cria o esquema, 0002_token_version adiciona a revogação de JWT e mudanças futuras viram novas revisões.
D11	Seed de demonstração	7 contas (1 personal e 6 alunos) com perfis de dados distintos (evolução boa — duas alunas —, estagnado, frequência irregular, evolução moderada e sem vínculo com solicitação pendente) para demonstrar alertas e relatórios. Contas: personal@trainup.com (código de convite TRAIN123) e aluno@, bruno@, carla@, diego@, elisa@ e fabio@trainup.com; senha de demonstração trainup123 (use SEED_DEMO=false fora da demonstração).
D12	IP do cliente e proxy confiável	O rate limit usa o IP da conexão; o cabeçalho X-Real-IP só é aceito com TRUST_PROXY_HEADERS=true, definido no docker-compose porque o back-end só é alcançável pelo nginx. Forjar o cabeçalho não burla o limite (RN08).
D13	Chaves estrangeiras no SQLite	database.py liga PRAGMA foreign_keys=ON no SQLite, igualando seu comportamento ao do PostgreSQL (por exemplo, exercício em uso numa ficha não pode ser excluído: 409).
D14	Integração contínua	GitHub Actions (.github/workflows/ci.yml) executa ruff, pytest, vitest, build do front-end e docker compose config/build a cada push e pull request.
D15	Passagem restrita	POST /automacao/passagem só é aceito do aluno liberado ou do seu personal; outro usuário recebe 403 (RN02).

6.3 Dificuldades e riscos tratados

Dificuldade / risco	Tratamento	Situação
Datas em UTC aparecendo no horário errado para o usuário	Conversão explícita local↔UTC nos formulários e agrupamento por dia local nos serviços (D01, RN15).	Tratado
Risco de recomendação de suplementos sem habilitação	Conteúdo trocado por dicas educativas com aviso (D02).	Tratado
Contraste insuficiente de cores da marca em botões e textos	Azul mais escuro para ações; cores vivas só em preenchimentos (D09).	Tratado
Demonstração da automação sem hardware	Catraca virtual e simulador de cenários, com identificação clara de simulação (D03, D04).	Tratado
Acesso indevido entre perfis e entre alunos	Função única de resolução de aluno por perfil, usada em todos os routers, com testes de 403/404 (RN02).	Tratado
Recuperação de senha sem serviço de e-mail	Token no log do backend; evolução futura (D06).	Aceito no MVP
Concorrência na catraca (duas leituras simultâneas)	Bloqueio de linha no PostgreSQL (SELECT FOR UPDATE) durante a operação crítica; no SQLite, o bloqueio é ignorado.	Mitigado no PostgreSQL; limitação mantida em SQLite.
Forjar o IP (X-Real-IP) para burlar o limite de tentativas de login	Cabeçalho confiável só atrás do nginx e teto por e-mail independente do IP (D12).	Tratado
Exclusão de exercício em uso passando no SQLite (chaves estrangeiras desligadas)	PRAGMA foreign_keys=ON no SQLite (D13), coberto por test_biblioteca_propria_de_exercicios.	Tratado
Confirmação de passagem por outro usuário	Verificação do aluno liberado ou do personal responsável (D15).	Tratado

6.4 Limitações conhecidas

- Não há envio nem verificação de e-mail; a recuperação de senha usa o log do backend.
- O rate limit de login é em memória (vale para 1 worker); com várias instâncias seria preciso Redis ou armazenamento compartilhado.
- O estado da catraca é persistido e, no PostgreSQL, a operação crítica usa bloqueio de linha (SELECT FOR UPDATE) para serializar leituras concorrentes da mesma catraca; SQLite ignora esse bloqueio.
- O Alembic possui baseline 0001 e migration 0002 para token_version; não há histórico de migrations anteriores ao baseline.
- O CRUD simples, a resolução de acesso e o conflito de agenda ficam nos routers; apenas indicadores, relatórios, resumo de sessão, PDF e a catraca estão em services.
- A sessão de treino em andamento não é retomada após recarregar a página (as séries já concluídas ficam salvas). O logout e a redefinição de senha por token invalidam os JWT anteriores do usuário; a alteração de senha por PUT /users/{id} (somente API) não os invalida.
- O painel de automação usa consulta periódica (2 s), não WebSocket/SSE.
- A interface não oferece edição do cadastro do aluno (PUT /users/{id}); o próprio aluno também pode remover a conta pela API (DELETE /users/{id}), sem tela correspondente. A edição de exercício não revalida nome duplicado, e GET /workouts/today sem o parâmetro weekday usa o dia da semana em UTC (a interface sempre envia o dia local).
- O PDF usa a fonte Helvetica padrão e gráficos do próprio reportlab.

- O front-end tem testes unitários e de componentes (vitest e Testing Library: utilitários, hooks, cliente da API, autenticação, roteamento, componentes de UI, Toast e Login), mas não há testes de interface ponta a ponta nem das telas de negócio (Agenda, Treinar, Catraca etc.). Os testes de back-end usam SQLite (o bloqueio de linha do PostgreSQL não é exercitado) e não executam as migrations Alembic, aplicadas pela aplicação na subida.
- Não há hospedagem de produção: a instância EC2 é usada apenas como ambiente de desenvolvimento (code-server).

7 Casos de Testes

A versão atual do código contém 61 testes automatizados de back-end (pytest, 95% de cobertura) e 64 de front-end (vitest, em 8 arquivos), e as verificações manuais de alguns requisitos não funcionais. Os testes ficam em backend/tests e frontend/src; as saídas da execução ficam em docs/evidencias (seção 7.3). A tabela lista os casos principais; a relação completa dos 61 testes de back-end está em docs/evidencias/backend-pytest.txt. Cada caso de teste segue o formato CTnnn – nome, com o requisito verificado e o resultado esperado.

7.1 Casos de teste

ID	Teste	Requisito / regra	Resultado esperado	Status
CT001	Cadastro de personal exige código (test_cadastro_de_personal_exige_codigo)	RF01 / RN10	Cadastro de personal sem código ou com código errado é recusado (403); com o código correto é criado.	Passou
CT002	Senha fraca e e-mail repetido são rejeitados (test_senha_fraca_e_rejeitada)	RF01 / RN10	Senha fora da política retorna 422; e-mail repetido retorna 400.	Passou
CT003	Limite de tentativas de login (test_rate_limit_do_login)	RF02 / RN08	Após 5 falhas por IP + e-mail (ou 20 por e-mail) em 5 minutos o login retorna 429.	Passou
CT004	Limite por e-mail com IP forjado (test_rate_limit_por_email_ignora_ip_forjado)	RF02 / RN08 / RNF01	Teto por e-mail vale mesmo trocando o IP (x-real-ip) a cada tentativa: 8 × 401 e depois 429.	Passou
CT005	X-Real-IP ignorado sem proxy confiável (test_x_real_ip_ignorado_sem_proxy_confiavel)	RF02 / RN08 / RNF01	Sem proxy confiável, o cabeçalho x-real-ip forjado não burla o limite de tentativas.	Passou
CT006	Recuperação de senha com token inválido ou senha fraca (test_recuperar_senha_token_invalido_e_senha_fraca)	RF03 / RN10	Forgot não revela e-mail inexistente; token inválido retorna 400; senha fraca no reset retorna 422.	Passou
CT007	Troca de senha derruba sessões antigas (test_troca_de_senha_derruba_sessoes_antigas)	RF03 / RNF01	Após redefinir a senha, o JWT anterior retorna 401 e o login com a nova senha funciona.	Passou
CT008	Vínculo com aprovação do personal (test_fluxo_de_vinculo_com_aprovacao)	RF04, RF05 / RN01	Solicitação por código fica pendente; só após a aprovação o aluno é vinculado.	Passou
CT009	Recusar solicitação não vincula (test_recusar_nao_vincula)	RF05 / RN01	Recusar mantém o aluno sem vínculo.	Passou
CT010	Personal não acessa aluno de outro (test_personal_nao_acessa_aluno_de_outro)	RF07 / RN02	Personal não acessa aluno de outro personal (404).	Passou
CT011	Remoção de aluno (test_remover_aluno_apaga_dados_e_nao_remove_personal)	RF07	Remoção de aluno retorna 204 e apaga os dados; remover conta de personal retorna 400.	Passou
CT012	Aluno não cria exercício nem edita o catálogo base (test_aluno_nao_cria_exercicio_nem_edita_catalogo_base)	RF08 / RN02, RN11	Aluno não cria exercício (403); catálogo base é somente leitura.	Passou
CT013	Biblioteca própria de exercícios (test_biblioteca_propria_de_exercicios)	RF08 / RN11	Criação na biblioteca própria (201), nome duplicado (400) e exclusão de exercício em uso (409).	Passou
CT014	Ficha de treino (CRUD) e ficha do dia (test_ficha_de_treino_crud_e_ficha_do_dia)	RF09, RF10	Criar, editar e excluir ficha; ficha do dia por weekday, com fallback para ficha sem dia; aluno não cria ficha (403).	Passou

ID	Teste	Requisito / regra	Resultado esperado	Status
CT015	Ficha só para aluno vinculado (test_ficha_so_para_aluno_vinculado)	RF09 / RN02	Personal não cria ficha para aluno de outro personal (404).	Passou
CT016	Sessão de treino com resumo e recorde (test_sessao_de_treino_resumo_com_recorde)	RF11, RF12 / RN13	Sessão registra séries e o resumo traz volume e recorde.	Passou
CT017	Série em sessão finalizada é rejeitada (test_serie_em_sessao_finalizada_e_rejeitada)	RF11, RF12 / RN12 / RNF06	Repetições acima do limite retornam 422; série em sessão finalizada retorna 409; finalizar de novo é idempotente.	Passou
CT018	Contagem de sessões sem recorde (test_streak_de_estagnacao)	RF14 / RN13	Contagem correta de sessões desde o último recorde.	Passou
CT019	Alerta de estagnação (test_alerta_de_estagnacao)	RF14 / RN13	Aluno sem recorde recente gera alerta com mensagem.	Passou
CT020	Dicas por objetivo com aviso (test_dicas_por_objetivo_com_aviso)	RF15 / RN14	Dicas do objetivo do aluno (3 dicas) acompanhadas do aviso educativo.	Passou
CT021	Relatório em JSON e PDF (test_relatorio_json_e_pdf)	RF16, RF18	Relatório em JSON e PDF são gerados.	Passou
CT022	Aluno só vê o próprio relatório (test_aluno_so_ve_o_proprio_relatorio)	RF16 / RN02	Aluno não acessa o relatório de outro aluno (403).	Passou
CT023	Observação do personal no relatório (test_observacao_do_personal_no_relatorio)	RF17	Observação aparece no relatório; acima de 2.000 caracteres retorna 422; aluno não grava observação (403).	Passou
CT024	Conflito de agenda (test_conflito_de_agenda)	RF19 / RN03	Aula sobreposta é recusada (409); aula em outro horário é aceita.	Passou
CT025	Transições de status da aula (test_transicoes_de_status)	RF20 / RN04, RN05	CHECKIN manual é recusado; aluno só cancela; aula cancelada não volta.	Passou
CT026	Check-in leva a aula a concluída (test_checkin_leva_a_concluido)	RF21, RF22 / RN04	Após o check-in, o personal conclui a aula.	Passou
CT027	Check-in na janela libera a catraca (test_checkin_dentro_da_janela_libera_a_catraca)	RF22 / RN06, RN09	Check-in válido percorre VALIDANDO → LIBERADA → AGUARDANDO_PASSAGEM e a aula vai a CHECKIN.	Passou
CT028	Segundo check-in da mesma aula é negado (test_segundo_checkin_da_mesma_aula_e_negado)	RF22 / RN06	Segundo check-in da mesma aula é negado (anti-fraude).	Passou
CT029	Check-in fora da janela é negado (test_fora_da_janela_e_negado)	RF22 / RN06	Check-in cedo demais é negado e a catraca permanece TRAVADA.	Passou
CT030	Passagem só pelo aluno liberado ou seu personal (test_passagem_so_pelo_aluno_liberado_ou_seu_personal)	RF23 / RN02	Passagem só é aceita do aluno liberado ou do seu personal (403 para os demais); sem pendência retorna 409.	Passou
CT031	Simulador de cenários (test_simulador_cenarios)	RF24, RF25 / RN16	Cenários válida, inválida, sem aula e dupla; estatísticas do dia.	Passou
CT032	Aluno não usa o simulador (test_aluno_nao_usa_simulador)	RF25 / RN02	Aluno recebe 403 ao usar o simulador.	Passou
CT033	Bloqueio após negativas seguidas e rearme (test_bloqueio_apos_negativas_seguidas_e_rearme)	RF22, RF26 / RN07	Três negativas bloqueiam a catraca; o rearme volta a TRAVADA.	Passou
CT034	Painel do personal (test_painel_do_personal)	RF27	Painel do personal retorna indicadores; aluno recebe 403.	Passou
CT035	Logout revoga o token (test_logout_revoga_o_token)	RF28 / RNF01	Após o logout, o JWT anterior retorna 401; novo login emite token válido.	Passou

ID	Teste	Requisito / regra	Resultado esperado	Status
CT036	Vitest — 8 arquivos (64 testes): utilitários de data/formatação, hooks, cliente da API, AuthContext, roteamento por perfil (App), componentes de UI, Toast e Login	RNF04 / RN15	Datas civis, semana iniciando na segunda e conversão America/Sao_Paulo → UTC corretas.	Passou
CT037	Autenticação — RNF01 (test_rate_limit_do_login, test_rate_limit_por_email_ignora_ip_forjado, test_senha_fraca_e_rejeitada, test_logout_revoga_o_token)	RNF01	Sem token ou token inválido → 401; senha fraca → 422; 6º login errado → 429; teto por e-mail → 429 (limite reduzido a 8 no teste); logout revoga o JWT.	Passou
CT038	Controle de acesso — RNF02 (test_aluno_so_ve_o_proprio_relatorio, test_aluno_nao_cria_exercicio_nem_edita_catalogo_base, test_personal_nao_acessa_aluno_de_outro)	RNF02	Aluno acessa relatório de outro aluno → 403; aluno cria exercício → 403; personal acessa aluno sem vínculo → 404.	Passou
CT039	Tempo de resposta da evolução — RNF03 (test_rnf03_evolucao_responde_em_ate_3_segundos e coletar_evidencias_api.py)	RNF03	5 chamadas da evolução em até 3 s. Medição de 24/10/2026: média de 18,5 ms e máximo de 51,5 ms.	Passou
CT040	Histórico incremental — RNF06 (test_serie_em_sessao_finalizada_e_rejeitada)	RNF06	Série em sessão finalizada → 409; progress e access_logs sem rotas PUT/PATCH/DELETE (inspeção do código e do Swagger).	Passou
CT041	Catraca — RNF08 (test_checkin_dentro_da_janela_libera_a_catraca, test_reset_fora_de_bloqueio_e_aluno_sem_permissao, test_rnf08_ciclo_da_catraca_dentro_de_1_segundo)	RNF08	Transições válidas e persistidas; rearme fora de BLOQUEIO → 409; cada ciclo em menos de 1 s (medição de 24/10/2026: 0 a 1 ms).	Passou
CT042	Hash de senha (test_senhas_do_seed_estao_em_bcrypt)	RNF01	Todas as senhas do seed em hash bcrypt ($2b$), nenhuma em texto puro.	Passou
CT043	Migrations Alembic (banco vazio → head; verificação manual, fora da suíte de testes)	RNF07	0001_baseline + 0002_token_version geram as 11 tabelas e a coluna users.token_version.	Passou
CT044	Build do front-end (vite build)	RNF07	Front-end compila sem erros.	Passou
CT045	Telas em desktop e celular (capturar_telas.py) e mensagem de erro do login (Login.test.jsx)	RNF04	26 capturas (13 telas × 1440 px e 375 px) sem rolagem horizontal; mensagem “E-mail ou senha incorretos” visível.	Passou
CT046	Configuração segura para nuvem (test_producao_exige_secret_key, test_producao_recusa_secret_key_curta, test_producao_aceita_secret_key_forte; só a porta do nginx exposta)	RNF05	RuntimeError sem chave ou com chave curta; chave forte aceita; segredos de produção vindos do ambiente.	Passou

Todos os requisitos funcionais possuem teste automatizado em pytest. CT036 a CT046 reúnem os testes de front-end e as verificações dos requisitos não funcionais; todos os casos passaram.

7.2 Resultado da execução

Execução de 24/10/2026 (Python 3.12.3 no back-end; Node.js v24.15.0 no front-end, na execução de 23/10/2026; o CI usa Python 3.12 e Node 22). Back-end: cd backend && python -m pytest -v → 61 testes aprovados, 0 falhas, cobertura de 95,47% das instruções; ruff com verificações aprovadas. Front-end: cd frontend && npm test → 64 testes aprovados em 8 arquivos (cobertura de 97,73% e npm run build sem erros na execução de 23/10/2026). Migrations 0001 e 0002 aplicadas do zero (11 tabelas), em verificação manual. Em 24/10/2026 também foram coletados, com os scripts de

docs/evidencias, o tempo de resposta da evolução (RNF03), o ciclo da catraca (RNF08) e as capturas de tela com a verificação de rolagem horizontal (RNF04). Os testes de back-end usam SQLite; o bloqueio de linha do PostgreSQL (SELECT FOR UPDATE) não é exercitado por eles (seção 6.4).

7.3 Registro das evidências

As evidências não são reproduzidas neste documento; ficam no repositório, principalmente na pasta docs/evidencias, com a seguinte organização:

README.md: como reproduzir os testes e onde estão as saídas.

backend-pytest.txt, backend-pytest-cobertura.txt e pytest_junit.xml: saída dos 61 testes de back-end, cobertura (95,47%) e relatório JUnit.

backend-ruff.txt: análise estática do back-end (ruff) sem apontamentos.

evidencias-api.txt e coletar_evidencias_api.py: medições de tempo de resposta (RNF03) e de ciclo da catraca (RNF08).

prints/, rnf04-overflow.txt e capturar_telas.py: 26 capturas de tela em desktop e celular e verificação de rolagem horizontal (RNF04).

frontend-vitest.txt, frontend-vitest-cobertura.txt e frontend-build.txt: testes (64), cobertura (97,73%) e build do front-end.

14-ec2-code-server.png: captura do code-server na instância AWS EC2 com a aplicação em execução (RNF05).

docs/api/: prints do Swagger (/api/docs) e openapi.json exportado, com a documentação da API.

docs/BACKLOG.md: backlog do produto atualizado.

CHANGELOG.md: registro de mudanças, decisões e mudanças de escopo do projeto.

8 Apêndices

Apêndice A — Dicionário de dados

Legenda: PK = chave primária; FK = chave estrangeira; UQ = único; N = aceita nulo. Enumerações são gravadas como texto e validadas em código.

Tabela `users`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	Identificador
name	VARCHAR(120)		Nome
email	VARCHAR(160)	UQ	E-mail de login
hashed_password	VARCHAR(200)		Hash bcrypt da senha
role	VARCHAR(10)		personal ou aluno
goal	VARCHAR(30)	N	Objetivo do aluno
personal_id	INTEGER	FK users, N	Personal do vínculo ativo
invite_code	VARCHAR(12)	UQ, N	Código de convite do personal
reset_token_hash	VARCHAR(64)	N	Hash do token de recuperação
reset_expires	TIMESTAMPTZ	N	Validade do token
created_at	TIMESTAMPTZ		Criação
token_version	INTEGER	NOT NULL, padrão 0	Versão do token; incrementada para revogar JWT no logout ou na troca de senha.

Tabela `link_requests`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
aluno_id	INTEGER	FK users	Aluno solicitante
personal_id	INTEGER	FK users	Personal destinatário
status	VARCHAR(10)		PENDENTE, ACEITO, RECUSADO, CANCELADO
created_at	TIMESTAMPTZ		Solicitação
responded_at	TIMESTAMPTZ	N	Resposta

Tabela `exercises`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
name	VARCHAR(120)		Nome
muscle_group	VARCHAR(60)		Grupo muscular
description	VARCHAR(500)		Descrição
video_url	VARCHAR(300)	N	Link de vídeo
owner_id	INTEGER	FK users, N	Dono; nulo = catálogo base

Tabela `workouts`

Coluna	Tipo	Restrição	Descrição
`id`	INTEGER	PK	
`name`	VARCHAR(120)		Nome da ficha
`personal_id`	INTEGER	FK users	Criador
`aluno_id`	INTEGER	FK users	Aluno
`weekday`	INTEGER	N	0=segunda a 6=domingo

Tabela `workout_items`

Coluna	Tipo	Restrição	Descrição
`id`	INTEGER	PK	
`workout_id`	INTEGER	FK workouts	Ficha
`exercise_id`	INTEGER	FK exercises	Exercício
`sets`	INTEGER		Séries
`reps`	INTEGER		Repetições
`target_load`	FLOAT		Carga-alvo (kg)

Tabela `workout_sessions`

Coluna	Tipo	Restrição	Descrição
`id`	INTEGER	PK	
`aluno_id`	INTEGER	FK users	Aluno
`workout_id`	INTEGER	FK workouts, N	Ficha (nulo = treino livre)
`started_at`	TIMESTAMPTZ		Início
`finished_at`	TIMESTAMPTZ	N	Término

Tabela `progress`

Coluna	Tipo	Restrição	Descrição
`id`	INTEGER	PK	
`aluno_id`	INTEGER	FK users	Aluno
`exercise_id`	INTEGER	FK exercises	Exercício
`workout_id`	INTEGER	FK workouts, N	Ficha
`session_id`	INTEGER	FK workout_sessions, N	Sessão
`set_number`	INTEGER		Número da série
`reps`	INTEGER		Repetições (1–200)
`load_kg`	FLOAT		Carga (0–1000)
`performed_at`	TIMESTAMPTZ		Execução

Tabela `schedules`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
personal_id	INTEGER	FK users	Personal
aluno_id	INTEGER	FK users	Aluno
start_time	TIMESTAMPTZ		Início
duration_min	INTEGER		Duração (15–240)
status	VARCHAR(12)		AGENDADO, CHECKIN, CANCELADO, CONCLUIDO
checkin_at	TIMESTAMPTZ	N	Hora do check-in

Tabela `report_notes`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
aluno_id	INTEGER	FK users, UQ	Aluno (uma observação por aluno)
personal_id	INTEGER	FK users	Autor
text	TEXT		Observação
updated_at	TIMESTAMPTZ		Atualização

Tabela `turnstiles`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
name	VARCHAR(80)		Nome
location	VARCHAR(120)		Local
state	VARCHAR(24)		TRAVADA, VALIDANDO, LIBERADA, AGUARDANDO_PASSAGEM, BLOQUEIO
state_since	TIMESTAMPTZ		Desde
unlock_until	TIMESTAMPTZ	N	Fim do tempo de passagem
blocked_until	TIMESTAMPTZ	N	Fim do bloqueio
consecutive_denials	INTEGER		Negativas seguidas
last_aluno_id	INTEGER	N	Último aluno (sem FK)

Tabela `access_logs`

Coluna	Tipo	Restrição	Descrição
id	INTEGER	PK	
turnstile_id	INTEGER	FK turnstiles, N	Catraca
aluno_id	INTEGER	FK users, N	Aluno
personal_id	INTEGER	N	Personal (sem FK)
schedule_id	INTEGER	N	Aula (sem FK)

Coluna	Tipo	Restrição	Descrição
event	VARCHAR(20)		TENTATIVA, PASSAGEM, TIMEOUT, BLOQUEIO, RESET
granted	BOOLEAN	N	Acesso permitido
reason	VARCHAR(200)		Motivo
scenario	VARCHAR(20)	N	Cenário simulado
simulated	BOOLEAN		Gerado pelo simulador
cycle_ms	INTEGER	N	Tempo do ciclo
detail	TEXT		Log do ciclo
created_at	TIMESTAMPTZ		Registro

Apêndice B — Tecnologias utilizadas

Principais tecnologias adotadas no TrainUp:

Camada	Tecnologia
Back-end	Python, FastAPI, SQLAlchemy 2, Alembic e Pydantic
Segurança	JWT (python-jose) e hash de senha com bcrypt
Relatórios	ReportLab (PDF)
Front-end	React 18, Vite, Tailwind CSS, React Router, Axios e Recharts
Banco de dados	PostgreSQL 16 (SQLite nos testes automatizados)
Execução	Docker Compose com nginx
Desenvolvimento	AWS EC2 com code-server (VS Code no navegador)
Testes e CI	pytest, Vitest e GitHub Actions

Apêndice C — Evidências dos testes

Execuções de 23 e 24/10/2026. A saída completa de cada comando está em docs/evidencias.

Suíte	Comando	Resultado
Back-end (pytest)	python -m pytest -v (em backend)	61 de 61 aprovados; cobertura de 95,47%
Front-end (Vitest)	npm test (em frontend)	64 de 64 aprovados (8 arquivos); cobertura de 97,73%
Back-end (ruff)	python -m ruff check app tests (em backend)	Verificações aprovadas
Front-end (build)	npm run build (em frontend)	Build concluído com sucesso